ITP: A Dependently Typed Proof Assistant with an Integrated Environment for Learning to Prove

Technical report, version 1

Aditya Ekbote

MIT World Peace University

Abstract

ITP is a proof assistant for a dependent type theory of the Calculus-of-Inductive-Constructions family, built together with an environment in which proving a theorem is a sequential decision problem. The system is organised around one rule: only the kernel decides what counts as a proof. The kernel, written in C++23, implements the calculus: a universe hierarchy with an impredicative bottom universe, dependent products, inductive families under strict positivity, structural recursion, local definitions, quotient types, and definitional proof irrelevance. Tactics, decision procedures (a ring normaliser, linear and nonlinear arithmetic by Farkas-style certificates) and proof search are untrusted programs that build terms; every finished term is re-checked by the kernel, and a second checker, written separately from the specification, re-checks the results. The environment makes a goal into data, lists the legal actions, probes each action on a restorable kernel state before taking it, keeps what an unfinished attempt did prove, and is exposed through the Gymnasium interface, all in one process and with deterministic budgets. This report specifies the calculus and the algorithms that decide it, including the pattern unification of metavariables; describes the implementation and its testing; and measures it on a two-core machine: a library of 555 declarations is built in 3.82 s and re-checked by the kernel in 0.417 s, and a tactic probe has a median latency of 0.607 ms. It states what is not done: soundness is tested, not proved, and a known gap in the admission of inductive families gives a proof of `False` once excluded middle and propositional extensionality are added to a world (Appendix A); the calculus has no cumulativity, universe polymorphism or mutual inductives; and no learning results are reported.

Keywords: interactive theorem proving; dependent type theory; proof checking; higher-order pattern unification; reinforcement learning environments; machine learning for theorem proving.

Contents

1 Introduction **3**

2 Theory: the calculus and the algorithms that decide it **4**
2.1 Conventions and configurations 4
2.2 Syntax 5
2.3 Typing 5
2.4 Reduction and conversion 7
2.5 Inductive families and case analysis 8
2.6 Structural recursion 10
2.7 Propositions and definitional proof irrelevance 11
2.8 Quotient types 11
2.9 Metavariables and higher-order pattern unification 12
2.10 The trust boundary 14
2.11 Certificate calculi 15
2.12 Metatheory: what is and is not established 16

3 Implementation **17**

3.1 The kernel . . . 17
3.2 The authoring layer and the libraries . . . 19
3.3 The independent checker . . . 19
3.4 The environment . . . 20
3.5 Testing . . . 23

4 Using the system **27**
4.1 Setting up . . . 27
4.2 A theorem, a decision procedure, and a refusal . . . 27
4.3 Integers, `linarith` and `nlinarith` . . . 28
4.4 An episode in the environment . . . 29
4.5 As a Gymnasium environment . . . 30
4.6 Checking, auditing, and reproducing . . . 30

5 What the design buys, and how it differs **31**
5.1 What a learning loop needs, and what the design provides . . . 31
5.2 Measurements . . . 32
5.3 How it differs from other systems . . . 35

6 Limitations **35**
6.1 The calculus . . . 36
6.2 Trust and testing . . . 36
6.3 The environment and the bank . . . 37

7 Conclusion **38**

A Two deviations of the kernel **40**
A.1 D1: the motive of a match is checked for its shape . . . 40
A.2 D2: strict positivity does not look in the indices . . . 41

B The reference proofs checked **45**

C Probes by kind of action **45**

D The gates **46**

1 Introduction

Machine learning for theorem proving asks a proof assistant to play two roles at once. It must be a *judge*: when a learned policy, a search procedure or a language model proposes a proof, the verdict has to be one that the proposer cannot influence, because the verdict is the training signal. And it must be an *environment*: a program that a learning loop calls a very large number of times, that presents a proof state as data, that can try an action and take it back, and that answers the same question the same way on every run. The two roles pull in different directions. A judge is valued for being small, stable, and independent of the cleverness around it; an environment is valued for speed, for introspection, and for the convenience of the interface it offers.

A common arrangement is to attach the learning system to a general-purpose proof assistant from the outside. The assistant runs as a separate process; a thin interface (a read–eval–print loop, a server, a set of bindings) exposes proof states and tactics; the learner sends text and receives text [4, 5, 20, 31, 45, 60, 61]. This arrangement is sound, and the field's results have been obtained with it. It also means that the environment inherits the properties of the assistant it wraps (its start-up cost, the cost of serialising a state, the granularity of its interface), and that what the learner can do in one step is whatever the assistant's surface syntax lets it say. A second line of work builds a small formal system for the purpose, so that the system and its environment are designed together [47, 48, 57]; the price is that the system is a fragment, not a general-purpose assistant.

ITP is a proof assistant and an environment designed together and run as one program. Its type theory is of the Calculus-of-Inductive-Constructions family [16, 17, 46]: enough to state and prove the mathematics of the competition-level and undergraduate problems we have formalised, and a good deal less than the full calculi of Lean or Rocq [10, 19, 54]. Its architecture follows from three decisions.

(i) One door. A declaration enters the global environment only through a checked entry point of the kernel, which type-checks the proof term against the statement in the empty context. The tactic engine, the decision procedures and the whole of the environment are on the other side of that door: they are untrusted programs that construct terms, and a bug in any of them can lose a proof but, so long as the kernel is correct, cannot make a false statement provable (Section 2.10).

(ii) Certificates, not oracles. The automation produces evidence that the kernel can check cheaply (a chain of ring-lemma applications; a nonnegative combination of hypotheses whose sum is a negative constant) rather than a verdict to be believed. A second checker, written from the specification and built differently, re-checks the results (Sections 2.11 and 3.3).

(iii) State is a value, work is a budget. Terms are hash-consed; a kernel state can be checkpointed and rewound in time proportional to what was allocated since; every function that can run long spends from a deterministic work budget and fails closed. An action can therefore be *probed* (tried, observed, undone) before it is taken, and the same probe gives the same answer on every machine (Section 3).

The environment (Section 3.4) is built on these. A goal is a record of hash-consed terms; the legal actions of a goal are listed as a finite menu (or said, token by token, with a typed term builder when they carry terms); each action is probed before it is taken; an attempt that stops half way still yields theorems (hindsight); and the whole is exposed through the Gymnasium interface [9, 55] with observations as arrays of token identifiers and no text anywhere.

Contributions. This report makes the following contributions; none of them is a claim of priority.

- A *specification of the calculus as implemented*, with the typing rules, the reduction and conversion relations, the admission conditions for inductive families, recursion and quotients, definitional proof irrelevance, and the algorithms that decide them, among them the pattern unification of metavariables after Miller [41] (Section 2).
- A *trust architecture* in which all automation is untrusted and every result passes one checked entry point, with a second checker for the results and a test apparatus aimed at the kernel (Sections 2.10 and 3).
- An *in-process proof environment*: goals as data, finite menus of legal actions, probes on a restorable

state, hindsight, a portable action format, and a Gymnasium interface, together with a bank of 268 problems of which 187 have kernel-checked reference proofs, and libraries of 555 declarations (Section 3.4).
- *Measurements* of what checking costs and how fast the environment is, on a two-core machine, and an account of what the system does not do (Sections 5 and 6).

What this report is not. It reports no results of learning: no model has been trained on the environment, and nothing here says how well a learner would do. It does not prove the system sound; the evidence is a test apparatus, described in Section 3.5, and the two checkers share one specification. It does not compare the speed of ITP with that of other systems: such numbers would be of different tasks on different hardware, and we give none. Where it contrasts design choices with those of other systems (Section 5), it does so qualitatively.

Outline. Section 2 specifies the calculus and the algorithms that decide it. Section 3 describes the implementation: term representation, evaluation, memoisation, resource control, the Python layers, the libraries, the independent checker and the testing. Section 4 shows how the system is used, with listings that were run to produce the output printed. Section 5 explains what the design buys for learning and how it differs from other systems, with measurements. Section 6 lists the limitations, and Section 7 concludes.

2 Theory: the calculus and the algorithms that decide it

This section specifies the type theory that the kernel implements and the algorithms that decide it. It is a specification of an implementation, not the presentation of a new type theory: every rule below corresponds to code in the kernel (Section 3) or in the independent checker (Section 3.3), and where the implementation does something other than the textbook rule we say so; the cases are collected in Appendix A. The calculus belongs to the Calculus-of-Inductive-Constructions family [16, 17, 46]. Its nearest relatives are the kernel calculi of Lean and Rocq [10, 52, 54]; the respects in which it is smaller are listed in Section 6.

2.1 Conventions and configurations

Typing is written $\Gamma \vdash t : A$ and definitional equality (*conversion*) $\Gamma \vdash t \equiv u$; the global environment Σ is fixed and left implicit. A *type* is a term A with $\Gamma \vdash A : \mathsf{Type}_i$ for some i. Substitution $t[x := s]$ is capture-avoiding, $\overline{a}$ abbreviates a sequence $a_1, \ldots, a_n$, and $f\,\overline{a}$ is iterated application. The implementation represents terms in a locally nameless style, with bound variables as de Bruijn indices and context variables as de Bruijn levels [12, 18, 40]; that is an implementation matter (Section 3) and plays no role in the rules.

The kernel has two configurations of the bottom universe, chosen when a world is created.

Set-like (`prop_elim` off). Type_0 is impredicative and proofs are terms like any others. An inductive family in Type_0 may not have a field in a larger universe; this is the condition under which an impredicative bottom universe admits strong elimination without paradox [14, 35] (rule A4 in Section 2.5).

Prop (`prop_elim` on). Type_0 is written Prop. It is impredicative and *proof-irrelevant* (Section 2.7). A family in Prop may have large fields, and one that is not a syntactic subsingleton can be eliminated only into Prop (Section 2.5). Quotient types are available (Section 2.8).

Every library and the whole problem bank use the second configuration, together with the option `parameters`, which makes the kernel distinguish a family's parameters from its indices (Section 2.5). Rules and conditions that belong to one configuration only are marked.

$$\frac{k < 2^{30}}{\Gamma \vdash \mathsf{Type}_k : \mathsf{Type}_{k+1}}\ \textsc{Sort} \qquad \frac{(x{:}A) \in \Gamma}{\Gamma \vdash x : A}\ \textsc{Var} \qquad \frac{(c : A) \in \Sigma}{\Gamma \vdash c : A}\ \textsc{Const}$$

$$\frac{\Gamma \vdash A : \mathsf{Type}_i \qquad \Gamma, x{:}A \vdash B : \mathsf{Type}_j}{\Gamma \vdash \Pi(x{:}A).\, B : \mathsf{Type}_{\mathrm{imax}(i,j)}}\ \textsc{Pi} \qquad \frac{\Gamma \vdash A : \mathsf{Type}_i \qquad \Gamma, x{:}A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ \textsc{Lam}$$

$$\frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\, a : B[x := a]}\ \textsc{App} \qquad \frac{\Gamma \vdash t : A \qquad \Gamma \vdash A \equiv A' \qquad \Gamma \vdash A' : \mathsf{Type}_i}{\Gamma \vdash t : A'}\ \textsc{Conv}$$

$$\mathrm{imax}(i, 0) = 0, \qquad \mathrm{imax}(i, j) = \max(i, j)\ \ (j > 0).$$

Figure 1. Typing rules, part one. The side condition of SORT keeps the successor of a level from wrapping around in the 32-bit counter that stores it. PI is the same in both configurations: the bottom universe is impredicative.

2.2 Syntax

Terms, which serve also as types and as motives, are generated by

$$\begin{aligned} t, u, A, B, M, s \ ::=\ & x \mid c \mid \mathsf{Type}_k \mid \Pi(x{:}A).\, B \mid \lambda(x{:}A).\, t \mid t\, u \mid \mathsf{let}\ x{:}A := t\ \mathsf{in}\ u \\ & \mid \mathsf{fix}_d\ f{:}A.\ t \mid \mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ b_1, \ldots, b_n \mid ?m \end{aligned}$$

where $k, d \in \mathbb{N}$ and c ranges over global names. The kernel has eleven node kinds; ten appear above, and the eleventh, a universe-parameter node, is scaffolding for universe polymorphism that is not used (evaluation refuses it). The term $\mathsf{fix}_d\ f : A.\ t$ is a local structural fixpoint that binds f in t and decreases on the d-th argument (0-based) of its type A (Section 2.6). The term $\mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ b_1, \ldots, b_n$ is case analysis on s with motive M; it has one branch per constructor of the family of s, in declaration order, and each branch is a curried function of *all* the fields of its constructor, parameters included (Section 2.5). The term $?m$ is a metavariable; it occurs during elaboration only, and no term admitted to the global environment contains one (Section 2.10). Constructors are ordinary constants. There are no primitive projections and no primitive recursors: recursion is fix together with match.

A context is $\Gamma ::= \cdot \mid \Gamma, x{:}A$. The global environment Σ is a list of declarations, each with a closed type:

- an *axiom* $c : A$;
- a *definition* $c : A := v$, theorems included; every definition is transparent to conversion, and there is no opacity attribute;
- an *inductive family* with its constructors (Section 2.5);
- a *recursive function* $f : A := t$ with a decreasing argument (Section 2.6);
- the five *quotient constants* (Section 2.8).

2.3 Typing

Figure 1 gives the rules for the variable, universe, dependent-product and application fragment, and Figure 2 those for local definitions, fixpoints and case analysis.

Universes. The hierarchy is $\mathsf{Type}_0 : \mathsf{Type}_1 : \mathsf{Type}_2 : \cdots$, with levels that are numerals. There is *no cumulativity*: a term of type Type_i does not have type Type_j for $j \neq i$, and a coercion between levels is an explicit construction. There is also *no universe polymorphism*: the calculus has no level variables, no level constraints and no level expressions. A development that needs a definition at several levels instantiates a construction function once per level vector, and the kernel checks each instance (Section 3). This is a deliberate restriction, taken to keep the universe fragment of the kernel trivially decidable and the independent checker small; its cost is discussed in Section 6. The side

$$\frac{\Gamma \vdash A : \mathsf{Type}_i \qquad \Gamma \vdash v : A \qquad \Gamma \vdash u[x := v] : C}{\Gamma \vdash \mathsf{let}\ x : A := v\ \mathsf{in}\ u : C}\ \textsc{Let}$$

$$\frac{\Gamma \vdash A : \mathsf{Type}_i \qquad \Gamma, f{:}A \vdash t : A \qquad \mathsf{guard}_d(f, t, A)}{\Gamma \vdash \mathsf{fix}_d\ f : A.\ t : A}\ \textsc{Fix}$$

$$\frac{\begin{array}{c}\Gamma \vdash s : I\,\overline{a} \qquad \Gamma \vdash M : \Pi(\overline{y}{:}\overline{Y})(z{:}Z).\,S \qquad |\overline{y}| = |\overline{a}| \\ \Gamma \vdash b_c : \mathsf{Br}(I, c, \overline{a}, M)\ \ (1 \le c \le n) \qquad \mathsf{Elim}(I, S)\end{array}}{\Gamma \vdash \mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ b_1, \ldots, b_n : M\,\overline{a}\,s}\ \textsc{Match}$$

Figure 2. Typing rules, part two. guard_d is the structural-recursion condition of Section 2.6. Br, the type of a branch, and Elim, the elimination restriction, are defined in Section 2.5. I is the family of the scrutinee, with constructors $C_1, \ldots, C_n$.

condition $k < 2^{30}$ in SORT is a guard against a specific failure: if levels wrapped around in 32-bit arithmetic, $\mathsf{Type}_{2^{32}-1}$ would be assigned the type Type_0, and a universe that contains itself admits Girard's paradox [14, 35].

Dependent products. The rule PI places $\Pi(x : A).\,B$ in $\mathsf{Type}_{\mathrm{imax}(i,j)}$. When the codomain is a proposition ($j = 0$) the product is a proposition whatever the level of the domain: this is impredicativity of the bottom universe [16], and it is what allows $\forall(A : \mathsf{Type}_5).\,P\,A$ to be a proposition. The kernel computes the level of a product telescope $\Pi(x_1{:}A_1) \cdots (x_n{:}A_n).B$ from the inside out, starting from the level of B and applying imax with the level of each A_j.

Application. The kernel types a whole application spine at once. The type of the head is exposed as a product telescope, by unfolding definitions at the head on syntax and, only if that fails, by evaluating and reading back; each argument is checked against its domain with the earlier arguments substituted; and the result type is obtained by substitution. Types are not normalised on the way. This is the rule APP iterated, with CONV applied where a domain must be matched.

Bidirectional operation. The rules are declarative; the kernel decides them bidirectionally. Inference synthesises a type for a term; checking verifies a term against a given type, and falls back on inference followed by conversion. Checking is where the two non-trivial elaboration steps happen: a match checked against an expected type gets its motive by abstracting the scrutinee and its indices from that type (Section 2.5), and a λ takes its expected domain from the expected product. Neither step is trusted: the term that results is checked in full, with the motive that was built.

Local definitions. LET types the body with the value substituted: the context never contains a definitional entry, and let is ζ-reduced rather than abstracted. The cost is that a body that uses its let-bound variable many times is re-typed against the value each time; the benefit is that contexts consist of assumptions only.

2.4 Reduction and conversion

Reduction is the union of the following rules, closed under all term formers.

$$
\begin{array}{ll}
(\beta) & (\lambda(x\!:\!A).\,t)\,u \;\rightsquigarrow\; t[x := u] \\
(\zeta) & \mathsf{let}\ x\!:\!A := v\ \mathsf{in}\ u \;\rightsquigarrow\; u[x := v] \\
(\delta) & c \;\rightsquigarrow\; v \qquad \text{if } c : A := v \in \Sigma \\
(\iota) & \mathsf{match}\ C_c\,\overline{v}\ \mathsf{return}\ M\ \mathsf{with}\ b_1, \ldots, b_n \;\rightsquigarrow\; b_c\,\overline{v} \\
(\varphi) & (\mathsf{fix}_d\ f\!:\!A.\ t)\ a_0 \cdots a_d\,\overline{a}' \;\rightsquigarrow\; t[f := \mathsf{fix}_d\ f\!:\!A.\ t]\ a_0 \cdots a_d\,\overline{a}' \qquad \text{if } a_d = C\,\overline{w} \text{ for a constructor } C \\
(\kappa) & \mathsf{Quot.lift}\ A\,r\,B\,f\,h\,(\mathsf{Quot.mk}\ A'\,r'\,a) \;\rightsquigarrow\; f\,a
\end{array}
$$

In (ι) the branch is applied to *all* fields $\overline{v}$ of the constructor, parameters included, which is why a branch is a curried function of all of them. The body of a recursive function is a λ-abstraction, so (δ) yields a value and does not diverge: a recursive call inside a branch is not evaluated until its branch is selected. The rule (κ) is the only reduction rule that is not β, δ, ζ, ι or the unfolding of a fixpoint (Section 2.8).

Conversion $\Gamma \vdash t \equiv u$ is the least congruence that contains $\rightsquigarrow^*$ and is closed under function η,

$$\lambda(x\!:\!A).\,t\,x \;\equiv\; t \qquad (x \notin \mathrm{FV}(t)),$$

and, in the Prop configuration, under definitional proof irrelevance (Section 2.7). There is no η for records or for the unit type, and none for a fix beyond the function η above.

Deciding conversion. Conversion is decided by normalisation by evaluation [2, 7, 26], in an environment-based, strict style. Terms are evaluated, in an environment of values, into a semantic domain $\mathcal{V}$:

$$
\begin{array}{rl}
v & ::= \ \langle \lambda(x\!:\!A).\,t, \rho\rangle \mid \langle \Pi(x\!:\!A).\,B, \rho\rangle \mid \mathsf{Type}_k \mid C\,\overline{v} \mid \langle \mathsf{fix}_d\ f\!:\!A.\ t, \rho\rangle \mid h\,\overline{\phi} \\
h & ::= \ \text{a context level} \mid \text{a global constant} \mid ?m \qquad \phi \ ::= \ \mathsf{ap}(v) \mid \mathsf{case}(M, \overline{b}, \rho)
\end{array}
$$

Closures pair a term with its environment ρ; a *neutral* term $h\,\overline{\phi}$ is a head applied to a spine of frames (an application, or a stuck case analysis that keeps its motive, branches and environment). The evaluator satisfies

$$
\begin{aligned}
\mathsf{eval}(\rho, t\,u) &= \mathsf{apply}(\mathsf{eval}(\rho, t), \mathsf{eval}(\rho, u)), \\
\mathsf{apply}(\langle \lambda(x\!:\!A).\,t, \rho\rangle, v) &= \mathsf{eval}((\rho, x{\mapsto}v), t), \\
\mathsf{eval}(\rho, \mathsf{let}\ x\!:\!A := v\ \mathsf{in}\ u) &= \mathsf{eval}((\rho, x{\mapsto}\mathsf{eval}(\rho, v)), u), \\
\mathsf{eval}(\rho, \mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ \overline{b}) &= \mathsf{elim}(\mathsf{eval}(\rho, s), \langle M, \overline{b}, \rho\rangle), \\
\mathsf{elim}(C_c\,\overline{v}, \langle M, \overline{b}, \rho\rangle) &= \mathsf{apply}^*(\mathsf{eval}(\rho, b_c), \overline{v}), \\
\mathsf{eval}(\rho, c) &= \mathsf{eval}(\cdot, v) \ \text{ if } c : A := v \in \Sigma, \qquad \mathsf{eval}(\rho, c) = c \ \text{ otherwise},
\end{aligned}
$$

together with the rules that saturating a constructor produces a value $C\,\overline{v}$, that a case frame is added to a neutral scrutinee, that Quot.lift fires (κ) when its sixth argument is a saturated Quot.mk, and that an application of a fixpoint value unfolds one layer. $\Gamma \vdash t \equiv u$ is decided by running $\mathsf{conv}_n(\mathsf{eval}(\rho_\Gamma, t), \mathsf{eval}(\rho_\Gamma, u))$, where $n = |\Gamma|$ and ρ_Γ maps each context variable to the neutral term at its level:

- two closures, or two product closures (after their domains), are compared by instantiating both bodies with the neutral term at the fresh level n and comparing at $n + 1$;
- a closure against a neutral term or a fixpoint is η-expanded: the other side is applied to the fresh neutral term and the results are compared;
- two constructor values must have the same constructor and pairwise convertible fields;
- two neutral terms must have the same head and spines of equal length whose frames are pairwise convertible; two stuck case frames must have the same motive term and branches that are convertible in their captured environments.

The comparison of stuck case frames and of fixpoints in captured environments is sound but incomplete, in the usual way: a pair that is convertible but was built in differently shaped environments may be rejected. When a spine comparison fails at a pair of frames that are not both applications, the failure is *uncertain*, and the comparison is repeated on the read-back normal forms of the two neutral terms; this recovers the cases that matter in practice, at the price of normalising. The environment-based evaluator is strict in its arguments; since the calculus is intended to be strongly normalising (Section 2.12), the strategy affects cost and not the relation. A fixpoint is unfolded one layer when it is applied, whatever the argument is, while the specification (φ) unfolds it only at a constructor; the independent checker follows the specification. The difference does not show in the artefacts we re-check: the independent checker accepts every global of every library and every reference proof of the bank (Section 3.3), and a proof that a tactic builds from a fixpoint wraps it in a barrier constant that keeps it folded (finding F23 in Section 3.5). In general, whether the eager unfolding can make the kernel accept a conversion that the specification does not is part of the gap discussed in Section 2.12.

2.5 Inductive families and case analysis

An inductive family is declared by its type and the types of its constructors,

$$I : \Pi(\overline{y}{:}\overline{\Xi}).\,\mathsf{Type}_l \qquad C_c : \Pi(\overline{x}{:}\overline{\Theta}_c).\; I\;\overline{q}_c \qquad (1 \le c \le n),$$

where $\overline{y}{:}\overline{\Xi}$ is the telescope of the arguments of I (parameters and indices alike), $\overline{x}{:}\overline{\Theta}_c$ are the *fields* of C_c (each type may depend on the earlier fields), and the *result indices* $\overline{q}_c$ are terms over the fields, one for each argument of I. A constructor takes its parameters as ordinary fields; for the list family, $\mathsf{cons} : \Pi(A{:}\mathsf{Type}_1)(a{:}A)(\ell{:}\mathsf{List}\,A).\,\mathsf{List}\,A$, whose result index is the bare field A.

Admission. A family is admitted by the checked entry point `finish_inductive` only if all of the following hold, for every constructor.

A1
The declared type of I is a type of the form $\Pi(\overline{y}{:}\overline{\Xi}).\,\mathsf{Type}_l$.

A2
Every field type is a type in the context of the earlier fields: $\Gamma_c, x_1{:}T_1, \ldots, x_{j-1}{:}T_{j-1} \vdash T_j : \mathsf{Type}_{s_j}$.

A3
The application $I\,\overline{q}_c$ is well typed and fully applied: $\Gamma_c \vdash I\,\overline{q}_c : \mathsf{Type}_l$.

A4
(*Universe rule.*) For every field x_j, one of the following holds: $s_j \le l$; x_j occurs as a bare variable among $\overline{q}_c$ (the field is *parameter-like*: this is how the kernel encodes a parameter, and a parameter is not a constructor argument in Lean or Rocq); or, in the Prop configuration, $l = 0$.

A5
(*Strict positivity.*) Every field type T satisfies $\mathsf{pos}_I(T)$, defined by $\mathsf{pos}_I(\Pi(z : Z).\,T') = (I \notin Z) \wedge \mathsf{pos}_I(T')$, and, for T not a product, $\mathsf{pos}_I(T) = (I \notin T) \vee$ (the head of the application T is I).

A6
The declaration is of a single family: mutual families are not available.

A4 is the condition that keeps the impredicative bottom universe consistent in the Set-like configuration: a constructor field of a larger universe in a family of Type_0 lets Type_0 be embedded in one of its own members (Girard's paradox [14, 35]); in the Prop configuration the same large fields are permitted, and the price is paid at elimination (below). **A5** accepts ordinary recursive fields ($\mathsf{Succ} : \mathsf{Nat} \to \mathsf{Nat}$) and infinitely branching ones ($\mathbb{N} \to \mathsf{Tree}$), and rejects $(\mathsf{Bad} \to \mathsf{False}) \to \mathsf{Bad}$; it rejects, too, an occurrence of I nested inside another family's argument (such as `List Bad`), because that would require the variance of the other family. We state the condition exactly as it is implemented because it is weaker than the textbook one in one respect: occurrences of I *inside the arguments of the head* I, that is, in the indices of a recursive field, and occurrences in the result indices $\overline{q}_c$, are not examined (Appendix A, D2).

Parameters. With the option `parameters`, the kernel computes a prefix of the arguments of I that are *parameters*: slot i is a parameter if, in every constructor, the i-th result index is a bare field variable different from the variables of earlier parameter slots, and every occurrence of I in any field type is fully applied and passes that same variable at slot i. A field that a parameter slot determines is treated as known in the branches of a case analysis (below); that is what makes the branch of cons in a match on a scrutinee of type List $\mathbb{N}$ mention $\mathbb{N}$ and not a fresh type variable. The independent checker re-implements this analysis from the stated rule.

Elimination into larger universes. In the Prop configuration a family in Type_0 is *eliminable only into* Prop (the family belongs to the set $\mathcal{E}$) unless it is a syntactic subsingleton: it has no constructor, or exactly one constructor each of whose fields is a proof (its type lives in Type_0) or occurs as a bare result index. This admits False, True, conjunction and equality with elimination into every universe, and restricts disjunction and the existential quantifier to eliminations whose motive ends in Prop; that is the usual rule of Coq and Lean, here stated on syntax. Families above Type_0 are data and are never restricted.

$$\mathsf{Elim}(I, S) \;:=\; \big(I \in \mathcal{E}\big) \Rightarrow S \equiv \mathsf{Type}_0.$$

Case analysis and the type of a branch. The rule MATCH (Figure 2) refers to the type of the branch for C_c, which we now define. Let $\overline{a}$ be the arguments of the family in the type of the scrutinee, $\overline{x}{:}\overline{\Theta}$ the fields of C_c and $\overline{q}$ its result indices. The kernel first matches the result indices against the arguments, producing a partial substitution ϑ on the fields (the *forcing*, below), and defines

$$\mathsf{Br}(I, c, \overline{a}, M) \;=\; \Pi(\overline{x}{:}\vartheta(\overline{\Theta}))\,.\; M\;\vartheta_{\mathrm{lin}}(\overline{q})\;\big(C_c\;\vartheta_{\mathrm{lin}}(\overline{x})\big).$$

Here $\vartheta(\overline{\Theta})$ applies ϑ to *every* field type, whereas ϑ_{lin}, used for the motive's arguments and for the constructor application, is ϑ restricted to the fields that occur at most once as a bare result index or that a parameter slot determined. The fields that occur twice (the field a of $\mathsf{refl} : \Pi(A{:}\mathsf{Type}_j)(a{:}A).\,\mathsf{Eq}\;A\;a\;a$) keep their own identity in the motive, which is what the usual elimination principle of equality prescribes. With $\vartheta = \emptyset$ the type is $\Pi(\overline{x}{:}\overline{\Theta}).\,M\,\overline{q}\,(C_c\,\overline{x})$, the textbook one.

Index forcing. Algorithm 1 is the first-order partial matching of the result indices of a constructor against the arguments of the scrutinee's family. A field that stands alone at an index position is forced to the corresponding argument; a field met again (as in (A, a, a) for equality) has its earlier value compared with the new one by the same procedure; two applications of the same constructor are matched argument by argument; two different constructors of one family *refute* the branch, since no constructor application can have both; and anything else is declined. Refutation is detected and not exploited: a refuted branch must still be present, and it is checked against the type obtained from the forcing computed up to the point of refutation. This is the point where the kernel departs from the elimination principles of Coq and Lean, where the branch type always has the generic indices and the equations between indices are left to the user: the aim is that a proof by cases on an indexed hypothesis can be written without an auxiliary equality in the common cases [25]. Because the calculus has uniqueness of identity proofs (Section 2.7), the forcing need not take the care that pattern matching without K requires [13]. The soundness argument is the one for the solution rule of dependent pattern matching: in any closed instance, the scrutinee is a constructor application whose fields satisfy the equations that the matching solved. The forcing is not itself a typing rule; its effect is only on the *expected* type of the branch, and the branch is then checked against that type in the ordinary way. We have not formalised the argument, and this is the most delicate part of the kernel; it is the part for which the independent checker (a re-implementation of this rule from the specification) and the mutation testing of Section 3.5 are the main evidence.

Motive inference. When a match is *checked* against an expected type T and its motive is a hole, the kernel builds the motive by abstracting the arguments and the scrutinee out of T:

$$M \;:=\; \lambda(\overline{y}{:}\overline{\Xi})(z{:}I\,\overline{y}).\; T[\overline{a} := \overline{y},\; s := z],$$

Algorithm 1 Index forcing: $\textsc{Force}_c(\overline{q}, \overline{a})$ for constructor C_c with result indices $\overline{q}$ over fields $\overline{x}$ and scrutinee arguments $\overline{a}$

Input: patterns $q_1, \ldots, q_r$ over the fields; concrete terms $a_1, \ldots, a_r$ over the context
Output: a partial map ϑ on the fields, or $\bot$ (the branch is *refuted*)

$\vartheta \leftarrow \emptyset$
for $i = 1, \ldots, r$ **do**
  if $\textsc{Match}(q_i, a_i)$ returns $\bot$ **then return** $\bot$
  end if
end for
return ϑ

function $\textsc{Match}(p, a)$
  if p is a field variable x_j **then**
    if $x_j \notin \mathrm{dom}(\vartheta)$ **then**
      $\vartheta(x_j) \leftarrow a$; **return** ok
    else return $\textsc{Match}(\vartheta(x_j), a)$ ▷ compare the earlier value with the new one
    end if
  else if p and a are applications with the same head constant and arity **then**
    return the first failure of $\textsc{Match}(p_k, a_k)$ over the arguments, else ok
  else if p and a have as heads two different constructors of one family **then**
    return $\bot$ ▷ refutation: the scan stops
  else return ok ▷ decline: nothing is forced, nothing is an error
  end if
end function

and checks that M is well typed; if it is not, the user must give the motive. When the motive is given, the kernel checks that its type has the shape $\Pi(\overline{y}{:}\overline{Y})(z{:}Z).\, S$ with $|\overline{y}| = |\overline{a}|$, and unifies $M\,\overline{a}\,s$ with the expected type. The motive's type is checked for its *shape* only; Appendix A (D1) states what is, and is not, compared.

2.6 Structural recursion

Recursion is by structural recursion on one argument of one non-mutual family. A recursive function is declared with a body $t = \lambda(p_0{:}P_0) \cdots (p_k{:}P_k).\, b$ and a *decreasing parameter* p_d whose type is headed by the family I; the local form is $\mathsf{fix}_d\ f : A.\ t$. Let $\mathsf{rec}(C)$ be the set of *recursive fields* of a constructor C of I: the fields whose type is directly an application of I (a field of type $\mathbb{N} \to \mathsf{Tree}$ is accepted by **A5** but is not recursive in this sense, so no call on it is accepted). The judgement $Sm \vdash e\ \checkmark$, where Sm is a set of variables known to be structurally smaller than p_d, is defined by the rules of Figure 3, together with the congruence rule that e is accepted when each immediate subterm is and f is not the term itself. A function is accepted, $\mathsf{guard}_d(f, t, A)$, if t has at least $d + 1$ leading abstractions and $\emptyset \vdash b\ \checkmark$.

The guard is the standard one for structural recursion [24] in its simplest form: the scrutinee must be a *variable*, only the direct recursive fields of a match on a smaller variable become smaller, a call must pass a bare smaller variable at the decreasing position, and no information flows through definitions, applications or non-variable scrutinees. A recursive function may use any function that is already complete (a constant like any other), but it cannot refer to itself except through the guard, and mutual recursion is not available. The kernel enforces the guard where a function is declared (`finish_recursive_function_checked`) and where a local fixpoint is typed (rule FIX); the independent checker re-implements it.

$$\frac{n \geq d \qquad a_d \in Sm \qquad Sm \vdash a_i \checkmark\ (0 \leq i \leq n)}{Sm \vdash f\ a_0 \cdots a_n \checkmark}\ \text{G-CALL}$$

$$\frac{s \in \{p_d\} \cup Sm \qquad Sm \vdash M \checkmark \qquad Sm \cup \{x_j \mid j \in \mathsf{rec}(C_c)\} \vdash e_c \checkmark\ (1 \leq c \leq n)}{Sm \vdash \mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ \lambda\overline{x}.e_1, \ldots, \lambda\overline{x}.e_n \checkmark}\ \text{G-MATCH}$$

$$\frac{s \notin \{p_d\} \cup Sm \qquad Sm \vdash s, M \checkmark \qquad Sm \vdash e_c \checkmark\ (1 \leq c \leq n)}{Sm \vdash \mathsf{match}\ s\ \mathsf{return}\ M\ \mathsf{with}\ \lambda\overline{x}.e_1, \ldots, \lambda\overline{x}.e_n \checkmark}\ \text{G-CASE}$$

$$\frac{y\ \text{fresh} \qquad Sm \vdash A, v \checkmark \qquad Sm' \vdash u \checkmark}{Sm \vdash \mathsf{let}\ y : A := v\ \mathsf{in}\ u \checkmark}\ \text{G-LET}$$

Figure 3. The structural guard. f is the function being defined and p_d its decreasing parameter; the field variables x_j of a branch are bound by its leading abstractions. In G-LET, $Sm' = Sm \cup \{y\}$ if v is a variable of Sm and $Sm' = Sm$ otherwise. A bare occurrence of f, or a call with fewer than $d + 1$ arguments, is accepted by no rule.

2.7 Propositions and definitional proof irrelevance

In the Prop configuration $\mathsf{Type}_0 = \mathsf{Prop}$ and proofs are irrelevant:

$$\frac{\Gamma \vdash P : \mathsf{Prop} \qquad \Gamma \vdash p : P \qquad \Gamma \vdash q : P}{\Gamma \vdash p \equiv q}\ \text{PI}$$

No other universe is irrelevant: two terms of a type in Type_j with $j \geq 1$ are equal only if they are convertible. The rule PI makes uniqueness of identity proofs, and the axiom K, provable by refl, and is also what lets a Boolean-valued statement be compared without comparing the proofs inside it (compare Gilbert et al. on definitional proof irrelevance without K [23]).

The rule is decided in the unifier only, and *after* the structural comparison has failed. A pair that the structural comparison did not equate is a *candidate* only if both sides pass a one-sided syntactic prefilter, which can say only "certainly not a proof"; this keeps failing comparisons of data, the common case in proof search, from paying for a type inference. The verdict then comes from the type checker itself: infer the type of each side, require that the type is a proposition (its sort is Type_0), and unify the two types. There is no hand-written classification of proofs that could drift from the typing rules. Partial effects of the failed structural comparison (metavariables assigned on the way to the mismatch) are rolled back before the rule is tried, so an equality obtained by irrelevance carries no commitments from a comparison that did not hold. The recursion (the verdict calls inference, which calls checking, which calls the unifier) is bounded by a nesting depth of eight, beyond which the answer is "not irrelevant": it fails closed. The untyped conversion conv of Section 2.4 does not apply PI: it never sees a type. Proofs inside the captured environment of a stuck case analysis are therefore compared structurally, which is incomplete and cannot be unsound.

2.8 Quotient types

A quotient is not a term former. It is five ordinary constants, whose types the kernel itself builds so that nobody can supply different ones, and the one reduction rule (κ). With $\mathsf{Eq} : (A{:}\mathsf{Type}_j) \to A \to$

$A \to \mathsf{Prop}$, where $j \geq 1$,

$$\begin{aligned}
\mathsf{Quot} &: (A{:}\mathsf{Type}_j) \to (A \to A \to \mathsf{Prop}) \to \mathsf{Type}_j \\
\mathsf{Quot.mk} &: (A{:}\mathsf{Type}_j) \to (r{:}A \to A \to \mathsf{Prop}) \to A \to \mathsf{Quot}\, A\, r \\
\mathsf{Quot.lift} &: (A{:}\mathsf{Type}_j) \to (r) \to (B{:}\mathsf{Type}_j) \to (f{:}A \to B) \\
&\quad \to \big(\Pi(a\,b{:}A).\, r\, a\, b \to \mathsf{Eq}\, B\, (f\, a)\, (f\, b)\big) \to \mathsf{Quot}\, A\, r \to B \\
\mathsf{Quot.ind} &: (A{:}\mathsf{Type}_j) \to (r) \to (P{:}\mathsf{Quot}\, A\, r \to \mathsf{Prop}) \\
&\quad \to \big(\Pi(a{:}A).\, P\, (\mathsf{Quot.mk}\, A\, r\, a)\big) \to \Pi(q{:}\mathsf{Quot}\, A\, r).\, P\, q \\
\mathsf{Quot.sound} &: (A{:}\mathsf{Type}_j) \to (r) \to (a\,b{:}A) \to r\, a\, b \\
&\quad \to \mathsf{Eq}\, (\mathsf{Quot}\, A\, r)\, (\mathsf{Quot.mk}\, A\, r\, a)\, (\mathsf{Quot.mk}\, A\, r\, b)
\end{aligned}$$

and (κ) as in Section 2.4. Making the primitives constants has a purpose: the hypothesis of Quot.lift, that f respects r, is then checked by the ordinary rule APP against the declared type, so there is no special typing rule that could forget to look at it. (In the displays above, (r) abbreviates $(r{:}A \to A \to \mathsf{Prop})$.) The standard soundness argument for Lean's quotients [10] applies: the respect proof is what makes (κ) agree with Quot.sound, and the motive of Quot.ind is a proposition because a motive in Type would let a function that does not respect r be defined by eliminating a quotient, after which Quot.sound would equate its values on related arguments. Quotients as a type former go back to Hofmann [30].

Declaring the quotient checks its preconditions: the Prop configuration; that the equality family is *exactly* the inductive family $\mathsf{Eq} : (A{:}\mathsf{Type}_j) \to A \to A \to \mathsf{Prop}$ with the single constructor $\mathsf{refl} : (A{:}\mathsf{Type}_j) \to (a{:}A) \to \mathsf{Eq}\, A\, a\, a$ (the constants' types mention it, and another family would give Quot.sound and the hypothesis of Quot.lift another meaning); that j, read off Eq, is at least one; and that no quotient has been declared before (a second would give the one reduction rule two meanings). The five types are built, type-checked, and only then declared; on any failure nothing remains declared. The independent checker does not trust the labels the kernel puts on these constants: it derives j and Eq from the declared type of Quot.sound and demands that every labelled constant have exactly the type above, up to α-equivalence. Function extensionality is not an axiom: it is a theorem of the library, derived from the quotient constants, and `audit` reports the quotient constants a proof depends on separately from axioms.

2.9 Metavariables and higher-order pattern unification

Tactics and the elaborator work with partially built terms. A goal is a metavariable $?m$ with a type and the context in which it was created; `apply` introduces a metavariable for each argument of a lemma that the goal does not determine; `exact` and `rewrite` leave constraints for terms that are not yet known. Resolving such constraints needs higher-order unification, which is undecidable in general [34]. The kernel restricts it, as every production elaborator does, to the fragment of Miller's *patterns* [41, 43], which has unique most general solutions, and defers everything else.

State. Metavariables live in a metavariable context: each has a type and, once assigned, a solution, a closed term. Assignment is recorded on a trail, so a context can be checkpointed and rewound in time proportional to what was assigned since. The elaboration state also holds a list of *pending* constraints, each a pair of terms with the context in which it was deferred. A term in which a solved metavariable occurs is read through the solution: a solved metavariable at the head of a neutral term is replaced by its solution before the term is compared.

Definition 1 (Pattern)**.** A neutral term is a *Miller pattern* if its head is a metavariable $?m$ and its spine consists of applications to *distinct bound variables of the context*, each a bare variable and not itself applied. In particular a pattern has no case-analysis frame in its spine.

For a pattern $?m\, x_1 \cdots x_k$ and a term t with $?m \notin t$ and $\mathrm{FV}(t) \subseteq \{x_1, \ldots, x_k\}$, the term $\lambda x_1 \cdots x_k.\, t$ is the most general solution of $?m\, x_1 \cdots x_k \equiv t$ [41]. Because $?m$ is replaced by a closed abstraction,

Algorithm 2 SOLVE$(m, \overline{x}, t)$: assign the pattern $?m\,\overline{x} \equiv t$ in a context Γ of size n

Input: a metavariable m unassigned; distinct context variables $\overline{x} = x_{l_1}, \ldots, x_{l_k}$ with types $D_1, \ldots, D_k$ taken from Γ; a term t (read back from a value)
Output: the assignment of m, or a failure

1: **if** m occurs in t **then return** failure ▷ occurs check
2: **end if**
3: $S \leftarrow \mathrm{FV}(t) \cup \bigcup_i \mathrm{FV}(D_i)$
4: **if** $S \not\subseteq \{x_{l_1}, \ldots, x_{l_k}\}$ **then return** failure ▷ scope: the solution may depend only on the spine, and so may the types of the spine
5: **end if**
6: let $y_1, \ldots, y_k$ be fresh context variables beyond Γ, $\varrho = [x_{l_i} \mapsto y_i]$
7: **for** $i = 1, \ldots, k$ **do**
8: $\quad D'_i \leftarrow \varrho(D_i)$
9: $\quad$ **if** D'_i mentions $y_i, \ldots, y_k$ **then return** failure ▷ the spine must be in dependency order
10: $\quad$ **end if**
11: **end for**
12: assign $m \leftarrow \lambda(y_1{:}D'_1) \cdots (y_k{:}D'_k).\ \varrho(t)$ ▷ closed over the fresh telescope
13: **return** success

the solution is well scoped wherever $?m$ occurs. In a dependent calculus the telescope of binders needs types, and SOLVE (Algorithm 2) takes them from the types of the spine variables, with two additional checks for the dependency that the simply typed setting does not have.

Unification. Algorithm 3 is the unifier. It extends the conversion of Section 2.4 with the two pattern cases. Two solved-or-unsolved applications of the same metavariable to the same arguments are equal (the reference kernel of Section 3.5 failed on $?m\,\overline{x} \equiv ?m\,\overline{x}$ because its occurs check fired on the problem itself). When both sides are patterns the left one is solved. A meta-headed problem that is not a pattern ($?m\,(g\,x) \equiv c$, or $?m\,x\,x \equiv c$ with a repeated variable) is neither solved nor failed: it is put on the pending list.

Draining. After a unification that may have solved a metavariable, the elaborator re-attempts every pending constraint in the context in which it was deferred, and repeats until a pass makes no progress (the number of pending constraints does not decrease) or the list is empty. A constraint that has become a pattern, or whose heads have been solved, is then decided. *Acceptance* of an elaboration requires that the pending list is empty: a tactic whose proof state is not fully resolved cannot be accepted, and a term whose constraints are not all decided cannot be admitted. This gate is not negotiable, because a deferred constraint is a promise that something will be compared, and a state that holds a promise it never keeps looks as if it had been checked.

What the algorithm is, and is not. Algorithm 3 is the pattern fragment of the textbook algorithm with the following departures, which make it weaker than a full implementation; none of them can cause an ill-typed term to be admitted, since the final term is checked in full.

- *No pruning.* A metavariable $?n$ inside t that is applied to variables outside the spine of $?m$ is not restricted to the variables of the spine: the scope check fails instead. Systems with pruning [1, 63] solve more problems.
- *No type check of the solution.* SOLVE checks scope and dependency order and does not check that the solution has the type of $?m$. The solution is type-correct whenever the problem was, because metavariables are created with the type of their context and every unification is of terms of one type; but the unifier does not prove this, and the elaboration of `apply` re-checks each solved argument metavariable on a scratch copy (finding F25 in Section 3.5). What is admitted is the *final* term, which is checked in full.
- *Patterns by construction.* The metavariable of a goal is applied to every variable of its context, hence

Algorithm 3 UNIFY(a, b) in a context Γ of size n, on values; the pending list and the metavariable context are state

1: $a \leftarrow \mathsf{resolve}(a)$, $b \leftarrow \mathsf{resolve}(b)$ ▷ replace a solved metavariable at the head by its solution
2: **if** a and b are meta-headed and $\mathsf{quote}_n(a) = \mathsf{quote}_n(b)$ **then return** success ▷ $?m\,\overline{x} \equiv ?m\,\overline{x}$
3: **end if**
4: **if** a is a pattern $?m\,\overline{x}$ **then return** SOLVE$(m, \overline{x}, \mathsf{quote}_n(b))$ ▷ the left side is tried first
5: **end if**
6: **if** b is a pattern $?m\,\overline{x}$ **then return** SOLVE$(m, \overline{x}, \mathsf{quote}_n(a))$
7: **end if**
8: **if** a or b is meta-headed **then**
9: push $(\mathsf{quote}_n(a), \mathsf{quote}_n(b), \Gamma)$ onto *pending*; **return** success ▷ deferred, not decided
10: **end if**
11: compare a and b by shape:
• two sorts: equal levels;
• two closures, or two products: UNIFY the domains, then the bodies applied to a fresh neutral term, in Γ extended by the domain;
• a closure against a neutral term or a fixpoint: apply the other side to a fresh neutral term (η) and UNIFY the bodies;
• two constructor values: same constructor, UNIFY the fields pairwise;
• two neutral terms: same head and spine length, UNIFY application frames pairwise; stuck case frames must have the same motive and branches convertible in their environments; on an uncertain failure compare read-backs;
• anything else: failure.
12: **if** the comparison failed **then**
13: roll back the metavariables and pending constraints it touched; try PI (Section 2.7)
14: **end if**

is a pattern; the restriction to patterns bites only on the metavariables of implicit arguments.

- *Deferral instead of search.* There is no backtracking over alternative solutions (no search for imitation and projection instances [34]); a problem that is not a pattern waits for other assignments or remains pending and refuses acceptance.

Example 1. In a context $x{:}\mathbb{N}$, $y{:}\mathbb{N}$ the problem $?m\,x\,y \equiv \mathsf{add}\,y\,x$ is a pattern problem: $S = \{x, y\}$, the dependency order is trivial, and the assignment is $?m := \lambda(x : \mathbb{N}).\,\lambda(y : \mathbb{N}).\,\mathsf{add}\,y\,x$. The problem $?m\,(\mathsf{Succ}\,x) \equiv c$ is deferred. The problem $?m\,x\,x \equiv c$, with a repeated variable, is deferred as well, and is decided only if $?m$ is assigned by another constraint.

Termination and backtracking. We do not argue termination of unification from a measure; it is enforced by the work budget of Section 3, which makes every call end in success, in failure, or in a budget error that fails closed. A metavariable is assigned at most once between a checkpoint and the rewind that follows it, and every tactic runs in a transaction that rewinds the metavariable context and the pending list on any exception, so that a failed attempt leaves no assignment behind.

2.10 The trust boundary

Admission to the global environment is the only operation that changes what is true.

Definition 2 (Checked entry points)**.** The global environment is extended by $\Sigma \to \Sigma,\ c : A := v$ only when

$$\frac{\Sigma;\cdot \vdash v : A \qquad \begin{array}{c} c \notin \mathrm{dom}(\Sigma) \qquad \Sigma;\cdot \vdash A : \mathsf{Type}_i \\ A, v \text{ contain no metavariable and no unfinished global} \end{array}}{\Sigma \ \longrightarrow\ \Sigma,\ c : A := v}\ \textsc{Declare}$$

and by the three other checked operations: `finish_inductive` (admission conditions **A1**–**A6**),

`finish_recursive_function_checked` (the body checked against its type under the guard), and `declare_quot` (the preconditions of Section 2.8).

The check of DECLARE is made in the *empty* context against a closed statement. Everything on the other side of this door (the tactic engine, the decision procedures, the elaborator and its unifier, the environment, the search procedures and the learner) is untrusted: it constructs candidate terms, and its only power over the environment is to submit them. A bug in any of it can fail to find a proof, or make a candidate that the kernel then refuses; it cannot extend the environment with an unchecked declaration.

The unchecked variants (`declare_unchecked`, `finish_inductive_unchecked`) exist, for axioms and so that the tests can build ill-formed environments on purpose. In the code we ship, the only call to one of them is the declaration of an axiom, and the libraries declare exactly one axiom, excluded middle, in the number-theory library. The function `audit` reports, for a theorem, the axioms and quotient constants that it reaches through the types and values of the definitions it uses, in the manner of Lean's `#print axioms`; a theorem that reaches none depends only on declarations that entered by a checked entry point, provided the discipline holds. The module that exposes the kernel to Python does expose the unchecked variants; the boundary is therefore a discipline of the layers above the kernel, backed by the second checker (Section 3.3), which re-checks the value of every definition in the exported environment, whichever path installed it. It does not re-check the admission of inductive families.

2.11 Certificate calculi

The decision procedures are *certifying*: each produces a proof term, built from lemmas of the libraries, that the kernel checks like any other. They are untrusted in the sense of Section 2.10: a wrong search produces a failure or a refused term, never a theorem. Each is described by the certificate that it searches for and the lemma chain that turns it into a term.

Ring normalisation. The tactic `ring` decides equality in the commutative semirings of the unary naturals Nat and the binary naturals N and in the commutative ring of the integers Z of the libraries, by normalising both sides to a canonical polynomial and building, at each step, the application of a ring lemma that justifies it [27, 53]. The normal form follows Mathlib's:

$$\text{poly} \;::=\; m_1 + (m_2 + (\cdots + (m_k + 0))) \qquad \text{mono} \;::=\; x_1^{e_1} \cdot (x_2^{e_2} \cdot (\cdots \cdot (x_j^{e_j} \cdot c)))$$

with monomials in strictly decreasing order, atoms in increasing index order, exponents $e \geq 1$ numerals and the coefficient c a numeral last. The order on monomials is the lexicographic order on exponent vectors, with the first atom most significant; it is total and compatible with multiplication, which is what lets a product of a monomial and a polynomial stay sorted without merging. Numerals are the canonical numerals of the number system, and arithmetic on them is checked by the kernel's evaluation (in time proportional to the number of bits for binary numerals). A power by a numeral uses repeated squaring, so $(a+b)^{20}$ takes five multiplications. In a ring, subtraction is multiplication by the numeral -1, and monomials whose coefficients cancel leave the sum. If the normal forms differ, the tactic fails and reports both.

Linear arithmetic by Farkas certificates. The tactic `linarith` reads each hypothesis as a fact $0 \leq p_i$ with p_i a polynomial over $\mathbb{Z}$ ($x \leq y$ becomes $0 \leq y - x$, $x < y$ becomes $0 \leq y - (x+1)$, an equality becomes two facts, a negated inequality is read as the reverse one), and negates the goal. It searches for integers $m_i \geq 0$ and $\kappa \geq 1$ such that

$$\sum_i m_i \, p_i \;\equiv\; -\kappa \qquad \text{as polynomials,}$$

every non-constant monomial cancelling, with a monomial that is not linear treated as an atom. The proof term is then: $0 \leq \sum_i m_i p_i$ by the lemmas `Z.mul_nonneg` and `Z.add_nonneg`; $\sum_i m_i p_i = -\kappa$ by `ring`; and the absurdity of $0 \leq -\kappa$ by `Z.not_nonneg_neg`. By Farkas' lemma, for a system of

affine non-strict inequalities over $\mathbb{Q}$ such a certificate exists if and only if the system has no solution, so the procedure is complete for such systems. It is incomplete over $\mathbb{Z}$: the facts $0 \le 2x - 1$ and $0 \le 1 - 2x$ have no solution in the integers and no certificate, since the procedure has no cutting planes, and the strengthening $x < y \Rightarrow x + 1 \le y$ recovers only part of integer reasoning. The search is a linear program (feasibility of $\sum_i m_i p_i = -1$ with $m \ge 0$ and a row per non-constant monomial), solved by an integer-preserving simplex method with Bland's rule [8] in exact arithmetic; for large problems a floating-point linear-programming solver (HiGHS through SciPy [32, 56]) first guesses the support, which the exact simplex then re-solves, so that floating-point arithmetic decides nothing.

Nonlinear arithmetic. `nlinarith` adds, before the search, the squares $0 \le t^2$ of the subterms $t \cdot t$ and t^2 occurring in the hypotheses and the products $0 \le p_i p_j$ of every pair of facts (at most 600), and then searches as above. The certificate is of the form $\sum_i \lambda_i p_i + \sum_{i \le j} \mu_{ij} p_i p_j + \sum_k \nu_k t_k^2 \equiv -\kappa$, a degree-two certificate in the style of the Positivstellensatz [29]. It is a heuristic, in the sense that the choice of products is a heuristic; any certificate it finds is checked.

Linear combinations of equations. `linear_combination` closes an equation $l = r$ from hypotheses $l_i = r_i$ and given coefficients c_i when $l - r = \sum_i c_i (l_i - r_i)$ as a polynomial identity. The proof applies the congruence lemmas of the ring to obtain $\sum_i c_i l_i = \sum_i c_i r_i$, proves $l + \sum_i c_i r_i = r + \sum_i c_i l_i$ by `ring` (rearranged so that no subtraction is needed), and cancels the common summand.

2.12 Metatheory: what is and is not established

We have not proved subject reduction, strong normalisation, consistency or canonicity for the calculus as implemented, and we have not mechanised any part of its metatheory. For the pure core, a dependent product with an impredicative bottom universe, a universe hierarchy without cumulativity, and the usual conversion, these results are known in the literature on the Calculus of Constructions and its extensions [6, 15, 16], and for inductive types under strict positivity and structural recursion in that on the Calculus of Inductive Constructions [17, 24, 46]. The kernel's calculus differs from those in features whose metatheory is not covered by such a reference, and each of these is a place where the implementation could be wrong in a way that no result we can cite excludes:

- the index forcing of Section 2.5, which changes the expected type of a branch;
- definitional proof irrelevance decided by the unifier and not by an untyped conversion (Section 2.7), with a depth bound;
- the strict-positivity check, which does not look in the indices (D2 in Appendix A, where a family that it admits yields a proof of False with two classical axioms);
- the shape-only check of a motive (D1);
- the eager one-layer unfolding of a fixpoint at any argument, against the specification's unfolding at a constructor (Section 2.4);
- the incompleteness and non-transitivity of conversion on stuck case analyses (Section 6), which can only reject and cannot accept an equation that evaluation does not justify.

Verified checkers exist for neighbouring calculi (MetaCoq for the kernel of Rocq, Lean4Lean for that of Lean [11, 52]); nothing of that kind exists here. The evidence that the implementation is sound is therefore a test apparatus: a second checker written from the specification, differential and randomised testing against a frozen reference kernel, mutation testing of the kernel and of the tools, sanitiser runs, and a findings log (Section 3.5). Such evidence finds bugs and, as the case of D1 and D2 shows, it also marks its own limits: both checkers share one specification, so a flaw in the specification itself is invisible to the comparison.

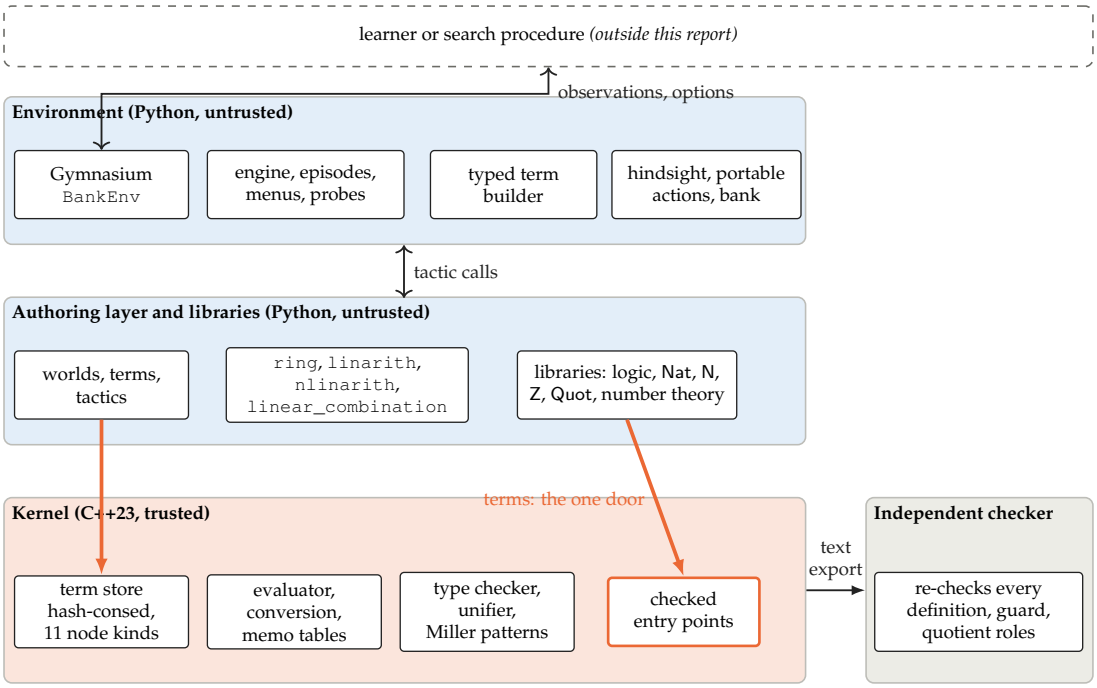


Figure 4. Architecture. Orange: the trusted base, the kernel, whose checked entry points are the only way a declaration enters the environment (the heavy arrows). Blue: untrusted layers; they construct terms and can only submit them. Grey: the independent checker, a second implementation of the specification that reads the kernel's text export of an environment and re-checks every definition.

3 Implementation

ITP is one program in two languages. The kernel is written in C++23 and exposed to Python through a nanobind module [36]; everything above it (the authoring layer, the libraries, the tactics, the decision procedures, the environment and the Gymnasium interface) is Python. Figure 4 shows the layers and the one direction in which trust flows, and Table 1 gives the size of each component at the commit at which the measurements of Section 5 were made.

3.1 The kernel

Terms. Every node is interned in a pool of its kind, an open-addressing hash table with a finalising mix of the hash so that structurally related nodes do not cluster; structurally equal terms are the same handle (a tag and an index), and equality of terms is a comparison of integers [22]. Terms are therefore DAGs, and the cost of checking a proof is a function of its size as a DAG and not as a tree. The values of the 516 definitions and theorems of the libraries have, together, 2,747,565 nodes counted as trees, whereas their types and values together consist of 132,822 distinct nodes, a factor of 20.7. Each compound node carries precomputed facts (the range of loose bound variables, the set of free context variables, a saturating size, and whether it mentions a metavariable), which let instantiation and abstraction skip every subterm they cannot affect. Bound variables are de Bruijn indices, so that a closed term has one representation, and variables of the context are de Bruijn levels, so that extending the context renumbers nothing [12, 18, 40].

Environments and values. The evaluator of Section 2.4 runs in persistent environments, singly linked lists with Myers' skew-binary jump pointers [42, 44]: extension is one allocation and indexing is logarithmic, and closures share their environments. Values are copied in constant time through shared immutable sequences, and the neutral-term spines of a conversion are compared by identity before they are compared by content. Reference counts are intrusive and released iteratively, since a chain half a million nodes long once overflowed the stack when it was freed recursively.

component	role	lines
kernel (C++23)	term store, evaluation, checking, unification, tactic core, bindings	8,288
frozen reference kernel	the previous generation, kept fixed as a point of comparison	11,046
authoring layer	worlds, terms, tactics, `ring`, `linarith`, `linear_combination`	2,256
libraries	logic, naturals, integers, quotients, number theory	3,453
independent checker	second implementation of the specification	848
environment	engine, menus, probes, term builder, hindsight, recorder	4,979
problem bank	268 statements, reference proofs, statement audit	8,943
Gymnasium and learning code	`BankEnv`; code of a first learner, not evaluated here	4,527
synthetic theorem generator		2,011
experiment drivers	benchmarks, measurement scripts, pipeline for later runs	5,592
tests (the gates)		25,433
mutation catalogue and driver		3,096
tools, examples		1,029, 569

Table 1. Size of the components in lines of source (Python, C++ and headers) in the files tracked by git at commit f840710.

Memoisation. Four tables and a quick check sit in front of the evaluator, and each table is keyed so that a hit is a fact about the question asked. The *typing memo* records the type of a term of at least 16 nodes, keyed by the node, the typing context, the evaluation switches and the epochs of the tables it depends on; it is used only for terms without metavariables in contexts without metavariables, and not at all while constraints are pending. A *quick conversion* on syntax runs before the evaluator and can only answer "yes" (identical nodes, congruence, binders, one delta step, beta), within a budget of steps and depth; on a "no" or a give-up the evaluator decides. Two tables remember pairs of shared constructor values and shared spines found convertible or unifiable after real work, and a fourth remembers the read-back of shared stuck values, which removes the exponential cost of failing on a DAG whose paths are all walked. During the build of the libraries, 46 % of typing queries are answered from the memo (89,691 hits, 106,411 misses), and the quick conversion settles 6,723 of the 17,972 comparisons it is asked. With a debugging switch on, *every* answer taken from any table is re-derived with all tables bypassed and a step limit, and a mismatch throws; the gates run with it on. No table stores a refusal, so a table can hide a bug only by answering "yes" wrongly, which is what the verification switch is for.

Resource control. Every function that can run long begins with a check that decrements a deterministic work counter ("fuel") and compares the stack pointer with the thread's floor, and throws a typed exception when the counter is spent or the stack is nearly full. Both exceptions are caught at the module boundary and turn a runaway computation into a refusal that names the cause; nothing else can be read from them, and no code path treats exhaustion as success. The authoring default is 2×10^9 steps per call; the environment gives each kind of action its own, much smaller, budget, so that a probe that does not finish is reported as having run out of work and not as having been refused. Conversions are nested at most 256 deep and proof-irrelevance checks eight deep, and beyond the bound the answer is "not equal" and "not irrelevant". The work of the decision procedures that is done in Python before the kernel sees a term is counted too, in units of the same budget (Section 3.5, finding F90).

State is a value. A *checkpoint* records the sizes of the node pools, of the table of globals, and the marks of the metavariable trail; *rewind* truncates the pools back to those sizes, removes the interned entries of the nodes it discards, and undoes the assignments since the mark. Its cost is proportional to what was allocated since the checkpoint. This is what a probe uses: run an action, observe the goals it leaves, and rewind, so that a refused action leaves the kernel exactly as it was. The Python layer

stage (module)	declarations	build, run 1 (s)	run 2 (s)
logic and equality (lnl_base)	40	0.00481	0.00675
unary naturals (nat_lib)	89	0.0540	0.0640
binary naturals (bin_lib)	102	0.170	0.182
ring lemmas over N (ring_lib)	21	0.0110	0.0109
quotients (quot_lib)	12	0.00196	0.00203
integers (int_lib)	170	0.346	0.320
number theory (nt_lib)	51	0.464	0.469
library extension (ext_lib)	64	2.34	2.24
olympiad problems (imo_lib)	6	0.423	0.527
total	555	3.82	3.83

Table 2. The library chain, built from nothing by ITP, on the machine of Section 5: declarations added at each stage and the time to build it, in two independent runs.

keeps handles to kernel nodes in caches that refuse to store while a probe is open, since a handle stored across a rewind could name a node that no longer exists or, worse, another node that took its index.

Determinism. No result depends on a memory address: tables are keyed by indices and hashes of indices, and no iteration order of an address-keyed container reaches a result. The same probe gives the same goals, and a whole run gives the same outcome digest on every repetition (Section 5).

3.2 The authoring layer and the libraries

The authoring layer builds terms and drives the kernel's tactics; *nothing in it is trusted*. A `World` is one kernel with a name table. Binders are written in higher-order abstract syntax (`w.pi(A, lambda x: B(x))`) and built in the locally nameless style: a bound variable is a fresh context variable while the body is built and is abstracted when the binder closes. Declarations (`inductive`, `define`, `recursive`, `theorem`) go through the checked entry points of Section 2.10. A proof is a script of tactics over named hypotheses (`intro`, `cases`, `induct`, `apply`, `rewrite`, `exact`, `ring`, `linarith`, and others); the script builds a term, and the term is submitted.

The libraries are built in stages, each on the last, entirely through this layer: logic and equality; unary naturals; binary naturals; the ring lemmas; quotients; the integers as the quotient of pairs of binary naturals; elementary number theory; an extension library; and the Olympiad-problem library. Table 2 lists the stages. A census of the 555 declarations gives 14 inductive families with 19 constructors, 31 definitions, 24 recursive functions, 461 theorems, the 5 quotient constants and one axiom, em (excluded middle, used by the number-theory library).

3.3 The independent checker

A second checker, `indep_checker`, re-checks the results. It was written from the specification of Section 2, not from the C++ kernel, and built differently on purpose.

	kernel	independent checker
language, size	C++23, 8,288 lines	Python, 848 lines (with its drivers)
variables	de Bruijn indices and levels	names, explicit capture-avoiding substitution
conversion	evaluation to a semantic domain, memo tables	weak-head normalisation, no caches
fixpoints	one layer on any argument	only on a constructor argument
η	function η	function η, in both directions
metavariables, let	supported	reported as unsupported

It shares with the kernel only the *input format*: the kernel's text export of an environment. It re-implements the universe rule for products, the structural guard, the parameter analysis, the MATCH rule with the forcing of Algorithm 1, the elimination restriction, definitional proof irrelevance with its depth bound, and the validation of the quotient constants; for those it does not trust the labels the kernel puts on them. It checks every definition and theorem of an environment (the value against the type), every recursive function against the guard, and every type; it does *not* re-check the admission of inductive families (the positivity and universe conditions **A4** and **A5**), nor the unifier, which it does not need: an admitted term contains no metavariable. Used on the libraries, it re-checks all 555 globals in about 82 s (after a load of 6.01 s) and accepts every one; on the reference proofs of the problem bank it re-checks a sample (Section 5). It is slow and memory-hungry by design: it is a witness and not a tool of the inner loop, and a proof it cannot finish within its limits is reported as such and counted, and not as accepted.

The checker has found defects in the kernel, among them an unvalidated binder annotation in a lambda checked against a product (F21); the comparison has also exposed disagreements that were defects of the checker (F27: the checker had no function η) and one defect of both, a branchless match on an inhabitant of an opaque type constant, which each accepted as a proof of anything until a test of quotients tried an inhabited opaque type (F26, a soundness defect, fixed in both). The last case is the lesson of the apparatus: two implementations of one specification agree on whatever the specification gets wrong.

3.4 The environment

The environment is the Python layer of `src/python/environment` together with the problem bank (4,979 and 8,943 lines, Table 1). It is untrusted (Section 2.10): it asks the tactic layer to construct terms, and what an episode ends with is a proof term that the kernel checks against the statement. Its job is to make three things operations of a library call and not services of a process: to read a state as data, to try an action and take it back, and to list what can be done.

Goals and steps. A *goal* is a target and the types and names of its hypotheses, all of them kernel handles. Terms are hash-consed, so two searches that reach the same goal by different routes reach the same object, and a goal has a key under which a search can recognise it. A goal can be read as arrays (node tags, scalars, child pointers, roots, heights) by a consumer that wants the term structure, or as a token sequence by one that wants a sequence. A *step* is an *action*, a kind and its arguments (Table 3), applied to the first open goal. After every step the goals it leaves are *settled*, in the same way in a probe as in a replay: a goal that a hypothesis or reflexivity closes is closed, and the quantifiers and implications at the head of a goal are introduced, so that no step is spent on either.

Probes. A *probe* takes a checkpoint (Section 3.1), runs an action on a one-goal copy of the goal, reads off the goals the action leaves, and rewinds: every node allocated in the meantime is reclaimed and every assignment of a metavariable undone. Its outcome is one of four statuses: `ok`, with the goals the action leaves; `refused`, when the tactic or the kernel said no, when the goal came back unchanged, or when the goals it leaves are not independent; `fuel`, when the work budget ran out; and `bug`, when something other than a refusal went wrong. Taking an action probes it first, so that an action that does not apply raises an error and changes nothing, and a learner or a search can try as many actions as it likes at the price of the probes.

Budgets. Every probe runs under a deterministic work budget that depends on the kind of action: 5×10^6 steps by default, 5×10^7 for `RING`, `LINARITH`, `INTERVAL_CASES` and `LINCOMB`, and 1.5×10^8 for `NLINARITH`. The decision procedures compute in Python before the kernel sees a term, and they spend the same units: the normaliser of `ring` counts its work before it allocates the memory the work stands for, and stops at one fiftieth of the kernel budget of the call. Listing a menu runs kernel calls as well (the normal forms of candidate facts and of the goal), and each is bounded by 5×10^6 steps, a candidate fact of `INSTANTIATE` by one fiftieth of that. A spent budget is reported as `fuel` and never as a refusal, since a larger budget might succeed; a learner should not be taught

kind	argument	effect
First language: the agent chooses an action from a finite menu (kinds 0–11)		
`INTRO`	—	introduce the first binder (needed only when automatic introduction is off)
`CASES`, `INDUCT`	a hypothesis, and hypotheses to generalise	case analysis, induction
`APPLY`	a lemma or hypothesis of the menu	apply it, where the goal determines its data arguments
`REWRITE`	a lemma or hypothesis, a direction	rewrite at the first instance
`RING`, `LINARITH`, `NLINARITH`	—	the decision procedures of Section 2.11; `NLINARITH` multiplies pairs of hypotheses
`BY_CONTRA`	—	classical: assume the negation, prove `False`
`INTERVAL_CASES`	a variable and its two bounds	split an integer variable bounded on both sides
`INSTANTIATE`	terms of the goal's pool	`have h := H` t_1 `...`
`USE`	a term of the pool	the witness of an existential goal
Second language: the arguments are said, one typed choice at a time (kinds 12–20)		
`CLAIM`	a statement	`have h :` `s`; two goals, the statement and the goal with `h`
`REFINE`	a term with holes	`exact`; each hole is a proposition left as a goal
`APPLY_WITH`, `REWRITE_WITH`	a source and the data arguments the goal does not determine	apply, rewrite; the unifier finds the rest
`REWRITE_AT`	as `REWRITE_WITH`, and a hypothesis	rewrite inside a hypothesis
`FORWARD`	a source and argument slots	`have h := source` s_1 `...`
`INDUCT_WITH`	a library principle, its other arguments, a variable	induction by a chosen principle; the motive is read off the goal
`INJECT`	a hypothesis $c\,\overline{a} = c\,\overline{b}$	the equations $\overline{a} = \overline{b}$
`LINCOMB`	hypotheses and coefficient terms	`linear_combination`, closed by `ring`

Table 3. The twenty-one kinds of action. The numbers are part of the data format of recorded proofs.

that an action is wrong because it was slow. The three places where such a bound was missing (the assembler of terms, the listing of a menu, the normaliser of `ring`) were found by long runs, two of which were killed by the operating system, and not by any gate; each is now pinned (findings F86, F87, F90 of Section 3.5).

The first language. Twelve kinds of action take their arguments from a finite menu. A menu is a list of actions together with the table of lemmas and hypotheses they name; the lemmas offered for `APPLY` and `REWRITE` are those that a library index says can match the goal, and for `APPLY` those whose data arguments the goal determines. The two kinds that carry terms, `INSTANTIATE` and `USE`, take them from a deterministic pool of small well-typed terms built from the variables and numerals of the goal. A menu is a finite, ordered, deterministic list, so that a policy can score it and a search can enumerate it.

The second language and the typed term builder. A finite menu cannot offer an arbitrary term: a witness, an intermediate statement, an instance of a lemma at chosen arguments. Nine further

kinds therefore take *codes*, tuples of tokens of a *typed term builder*. A term is built in prefix order, one choice per step, and at each step the builder offers only the tokens that keep the term well typed: a variable of the context applied to k arguments, a global applied to k arguments, a numeral of the type at the hole, a lambda, a universal quantifier, an implication, a sort, a term of the pool, or (for `REFINE`) a proposition left open as a goal. A head takes the arguments its type needs to reach the type at the hole, except those that the expected type already determines (the implicit carriers), so that a statement is said without the noise that a fully applied term would carry. A policy that chooses among the offered tokens cannot say an ill-typed term; the gates check this by random walks over the choices, and the kernel checks the finished term again. The builder says and builds back the terms of the proofs of the libraries (`encode`, `build`) with one exception that it reports and does not hide: a term with `match` or `let` cannot be said, which concerns two proofs of the libraries and the statement of functional extensionality. A lemma above the engine's premise limit cannot be said either.

Premises and regimes. An attempt may use only the globals that its *premises* allow, a number: a theorem is attacked with the library that precedes it, and the problems of a bank do not see each other's solutions. Two regimes follow from the lemmas that were written for a problem. In the *prefix* regime every global before the theorem is on offer, the problem's own helpers included, which measures how well an agent uses a good lemma; with its helper on the menu a problem can be a single action. In the *strict* regime the helpers written for the problem are hidden, from the menu and from the builder alike, so that the problem tests whether the ideas can be found. A reference proof must be self-contained for the strict regime (it proves its lemmas inline), and this is checked by replaying the proof with the helpers hidden, not assumed. An episode carries its own premises, so that episodes of different regimes interleave on one engine.

Portable actions and the recorder. An action can be written as plain data, with no kernel handle and no index into a menu: a lemma is named by its identifier in the world, a hypothesis by its level. A recorded proof can therefore be stored in a file, sent to another process and replayed in a fresh engine; malformed data raises the environment's error and nothing else. The *recorder* turns authored tactic proofs into such sequences. It runs a library with every tactic call logged as a tree, translates each call into candidate plans of engine actions, keeps the first plan whose probe leaves as many goals as the author's call left, carries it out, and finally has the kernel check the proof the engine built. All but six of the 459 recorded proofs of the libraries replay in this way; the six, named in the gate S40 together with their reasons, are two proofs whose terms contain a `match` or compute only, two authored inductions whose hypothesis the engine does not have, and two applications whose goals the engine settles by itself. A translation that is wrong makes a replay fail, and cannot produce a false theorem.

Hindsight. A failed attempt contains theorems that *were* proved: a case that closed, a fact that an action derived. With tracking on, an episode keeps its proof tree and returns these as new standalone problems with proofs, by analogy with relabelling a failed trajectory by what it did achieve [3]. Because the engine settles goals, the statement of a sub-tree as the agent saw it is not always the statement that a replay proves; an example is therefore returned only after it has been replayed from its own data in a fresh engine and checked by the kernel, and the candidates that cannot be made exact are dropped and counted by reason. Hindsight is a feature of the data, not of the logic: the examples are as sound as the replay that produced them.

The problem bank. A *problem* is a theorem of one deterministic world plus what a dataset needs to know about it: the problem in words, its source, topics and a level from 1 to 5, whether it asks for a proof, for an answer ("determine all": the answer is part of the statement, an equivalence with the solution set) or for an example, a reference proof, and an executable reading of the intended meaning. The bank has 268 problems from AMC, AIME, IMO, ISL and APMO papers, classical exercises, and graded problems written for the bank, in a world of 748 globals (the libraries and the statements). 187 of them have a reference proof that is kernel-checked and replays through

the action language; the remaining 81 are *statement-only*: the formal statement is the task, and an agent that proves one is checked by the kernel all the same. The reference proofs total 6,033 actions (median 11, 90th percentile 70, longest 466). The kernel establishes that a proof proves its statement; that the statement says what the problem says is a separate question. A statement is therefore read back from its kernel term into a small syntax tree and evaluated on integers, over windows of small values, against an intended meaning that the person who formalised the problem wrote as a Python predicate; the meaning of each symbol of the library is written down once, in a few dozen lines to audit, and compared with the library's definition. Mutated statements (a swapped relation, a dropped hypothesis) serve as negative controls that the comparison must notice or explain. A check on a window is a check of meaning on small values and never of truth. For the statement-only problems, an exhaustive search on a larger range, written from the problem and not from the formal statement, supplies evidence for the claim itself. A statement audit reports, for every problem, the checks made and the cautions where the evidence is weaker. No second reader's verdict is recorded yet, and a misreading of the English that the statement and its executable reading share is invisible to every check (Section 6). The splits into training, development and test problems are by family and not by problem (the two parts of one olympiad problem, or a statement and its restatement, belong together), and the split of a family is a pure function of its key, so that adding problems moves none.

The Gymnasium interface. `BankEnv` wraps the engine for the bank as a Gymnasium environment [9, 55] at the level of the policy's *decisions*. One episode is one proof attempt on the first open goal. To act, the agent answers the engine's questions in turn (the kind of action, the lemma, the hypothesis, then the next token of a term), each among the *options* that the engine offers at that point; when the last question is answered the action is probed and, if it applies, taken. The observation is a dictionary of integer arrays: the goal as token identifiers with a segment and a hypothesis index for each token, the number of hypotheses and goals left, the budget left, the overlap of each global's symbols with the goal, the kind of decision, the tokens of the type of the term being said, and the options as integer rows with a count and a legality mask. The action space is `Discrete(1024)`, the index of an option; an illegal index raises an error, or with `illegal='penalize'` loses the decision. The reward is 1 when the finished proof has been checked by the kernel, and an optional penalty per action or refusal; an episode is truncated after 60 actions (taken or refused) or 600 decisions, both configurable. An episode may start from any closed statement given as portable data, which is how hindsight examples are used. There is no text interface anywhere in this chain.

3.5 Testing

The architecture localises what must be right. Only the kernel can accept a false statement; the independent checker is a witness whose disagreement is information; everything else can lose a proof but not make one. The tests are accordingly aimed at the kernel and at the boundary around it, and the rest of the apparatus protects usefulness (an action that should apply, a probe that returns). This subsection describes the apparatus and states what each part shows. The numbers are those that the named gates printed when we ran them for this report (`measure_gates.py`), at commit f840710 on the machine of Section 5.2; all 54 gates were then run once more, as a whole (Appendix D), and the logs of both runs are in the supplementary material.

Gates. A *gate* is a self-contained script that exits 0 and prints no failure if and only if the property it protects holds. There are 54 (49 by default; the others need the machine-learning libraries, the repository, or the synthetic theorem generator), organised by what they protect: the kernel against a frozen reference, its soundness, boundaries and performance guards, quotients, proof irrelevance and the work budget; the libraries and decision procedures; the environment; the problem bank; and the repository itself. The gates of the default set can also be run with every answer that the kernel takes from a sharing table re-derived with the tables bypassed (`ITP_VERIFY_MEMO=1`, Section 3.1); this about doubles their time and is how the memo tables are tested.

apparatus	what is compared or attempted	result at this commit
lockstep against the frozen reference kernel (S2, harness)	random typed terms with `match`, corpora of inductive families and recursive functions, random terms for normal forms, conversion and typing; both kernels must give the same verdict	1,500 well-typed and 750 ill-typed matches, 1,503 dependent-typing checks, 12 admission and 6 guard cases, 9,500 comparisons on 1,500 terms; no disagreement
self-consistency oracle	random tactic scripts in five distributions; after each tactic the partial proof must type-check against the statement	504 proofs completed, 504 verified; no unattributed failure
independent checker (S8)	random scope-aware mutations of valid proofs, judged by the kernel and by the checker	3,043 mutated proofs: both accept 137, both reject 2,906, no verdict differs
soundness canaries (S15, S16)	a blind random generator of proof attempts on statements that are *false*; the same generator on true statements as a check on its power	classical axioms: 2,100 attempts at 7 false statements, none completes; 21 of 1,200 attempts at true ones complete, all verified. Quotients: 4,200 attempts, none completes; 67 of 3,000, all verified
boundary (S19)	malformed or hostile calls through the Python API: nothing may crash, hang, or be accepted	188 cases pass; 4 seeds of 15,000 random calls, each in a child process

Table 4. Parts of the test apparatus aimed at the kernel and what they printed. A random generator that completes nothing is also what a toothless generator does, so each canary is paired with a run of the same generator on true statements, which must complete and verify a minimum number of proofs.

Lockstep against a frozen reference. The previous generation of the kernel (absolute universe levels, variables as levels) is kept byte-for-byte, with its checksums pinned by a gate, and the same random inputs are given to it and to the hybrid kernel. Where the hybrid kernel is deliberately stricter, or fixes a defect of the reference, the gate says so explicitly (for instance F19; Table 5 counts the findings by layer). The lockstep comparison tests that a rewrite of the representation preserved the behaviour; it says nothing about a defect that both kernels share.

A second checker. Section 3.3 describes the checker; Table 4 gives its agreement with the kernel on mutated proofs. The mutations (a variable replaced by another in scope or by a constant, a constant replaced, a universe level shifted, the two parts of an application exchanged, a binder annotation replaced, two branches of a match exchanged, the decreasing argument of a fixpoint moved) produce mostly ill-typed terms, which is what the comparison needs: a checker that accepts a term that the kernel rejects, or conversely, is a finding.

Canaries. If a rule added to the trusted base made it inconsistent, a blind generator of tactic scripts would eventually complete and verify a proof of a false statement. The canaries of the two additions that put the base at risk (the classical axioms, in the world in which `Type 0` is `Prop`, and the quotient constants) throw random attempts at statements that are false in the intended model. That none completes is evidence of the kind that testing gives: of the absence of a cheap attack, not of consistency.

Falsification through the environment. A fuzzer drives the tactic layer, the unifier and the kernel's final check through the environment: it generates statements about the natural numbers or the

integers, decides by evaluation which are true (true ones by equality-preserving rewrites of a random term, false ones by a small perturbation of a true one), and runs the search on each. It classifies the outcomes as `REJECTED` (a finished proof that the kernel then refused: a defect of the tactic layer or the unifier), `UNSOUND` (a proof of a statement that is false at some point: a defect of the kernel, which should be impossible) and `ERROR` (any other exception out of the engine). A rejection is always a bug, since it is a disagreement between two layers; the first two defects found this way were in the tactic layer and the unifier. The campaign that collects its results into one folder (`make soundness`) takes hours, was not run for this report, and its output is not quoted.

Mutation testing. A test suite that has never failed has not been shown to test anything. The mutation catalogue holds 1,707 mutants, each of which re-breaks one rule of the system in an isolated copy of the tree, and the gates must fail on it. Of these, 663 mutate the C++ kernel (568 the header, 95 the bindings), 39 the independent checker, 252 the tactic layer and the decision procedures, 24 the libraries, and 729 the environment, the action language, the term builder and the recorder. A mutant that no gate catches is a gap in the tests unless it is one of the 22 documented equivalents, for which no input can tell the mutant from the original. Its limits are part of the result. The catalogue was run series by series as the code it refers to was written; the more recent additions were run against the gates that pin them; we report no single pass of all of it against the final commit, which takes many hours. There are no mutants of the same kind for the problem-set modules, the learner, the generator or most of the experiment pipeline, whose gates rely on fixtures with hand-worked expectations and on tamper tests, and have not been shown to bite by mutation. Some mutants are known to survive (one of the recorder's, whose fallback nothing exercises), and twenty environment mutants whose anchors were repaired by hand after a refactoring were not run again. The mutants are written by the author of the code, so they measure the tests against the faults that the author can imagine.

Sanitizers. The evaluator owns its memory by hand (intrusive reference counts, single-allocation sequences, a trampoline that frees deep chains iteratively), and a bug there would not show as a wrong answer but as a use after free that the functional gates cannot see. The kernel can be built with AddressSanitizer and UndefinedBehaviorSanitizer, and the gates run under it. For this report we ran 29 of the 38 gates that the tool selects by default (the lockstep, soundness, quotient, irrelevance, conversion, tactic and independent-checker gates of the kernel and the differential harness) with the sanitizers on, and all pass, in 305 seconds. The tool leaves out 11 further gates, because they limit the address space, which the sanitizer cannot start under, or carry time limits meant for an unsanitized build, or recurse a gigabyte deep, which the sanitizer reports as a false positive. The remaining gates of the default set, the long ones of the libraries, the environment and the bank, were not run under the sanitizers. Leak detection is off, since the Python interpreter itself leaks at exit; the evaluator's leaks are checked separately, by a standalone driver, which we did not run for this report.

Faults in the tools that make evidence. The numbers of an experimental report come from tools, and a test written together with the code it checks tends to check what the code does and not what it must do. Single-line faults were therefore injected into each of 6 tools (the soundness folder, the statement audit, the number checker, the record of which code produced a result, the digest of the code, and the runner of these faults itself), 252 faults in all, and the gate section written for the tool must fail for every one. Of the first 172 faults, 21 got past the tests that had been written for the tool, among them stages that reused a saved result made for other code and would have made a folder say `CLEAN` about code it had never looked at (finding F88); all 252 are now caught.

The log of findings. Every defect found while building the system, by a gate, a mutant, the second checker, a long run or a reading, is recorded in a log with its status and the test that fails without the fix. It holds 91 numbered findings (92 rows; the second, F2, has two parts). Of the rows, 76 are fixed and pinned by a test that fails without the fix, 5 are removed by the construction of the hybrid representation, 2 are mitigated with the root cause open, 1 is partial, 4 are documented design consequences or hazards, and 4 are open (Section 6). The author tagged each row, when it was

layer	trust	rows	examples
kernel	trusted	25	a lambda checked against a product never validated its binder annotation (F21); evaluation unfolds a fixpoint on a neutral argument (F23); conversion is incomplete on stuck matches (F28); work unbounded (F30)
independent checker	witness	5	no function η (F27); a branchless match on an opaque type constant accepted (F26)
tactic layer, unifier, libraries	untrusted	24	`apply` accepted a solution of the wrong type for a metavariable (F25); a work bound missing in `ring` (F90)
environment: engine, builder	untrusted	16	unbounded kernel calls while listing a menu (F87) or assembling a term (F86)
bank, recorder, hindsight, generator	untrusted	15	statements, duplicated or misleading data; exactness of hindsight (F72)
training and experiment pipeline	untrusted	7	estimates, documents, the record of the code
tests and tooling	—	4	the mutation driver was vacuous after a reorganisation: every mutant looked caught (F71)

Table 5. The 92 rows of the log of findings by the layer in which the defect lived. Three rows record trusted code that accepted what it must refuse (F19, F21, F26): the first is in the frozen reference kernel (the hybrid kernel is stricter), the second was found by the independent checker, and the third, a soundness defect found while designing the quotient gate, was in the kernel and in the checker. All three are fixed and covered by tests that fail without the fix.

written, with the layer in which the defect lived and the effect it could have had; Table 5 counts them (a row with two layers counts in both).

What the lessons are. Four findings are lessons about the apparatus and not about the logic. *Two implementations of one specification agree on whatever the specification gets wrong* (F26): a branchless `match` on an inhabitant of an opaque type constant was accepted as a proof of anything by the kernel and by the checker, because both read the family of a type from the same unwritten assumption, and it was found by a test of quotients that for the first time inhabited an opaque type. *A check that can fail for reasons of infrastructure needs a baseline that runs in the same infrastructure* (F71): after a reorganisation, the copy of the tree in which the mutation driver runs a mutant lacked a directory, every gate crashed on an import, every mutant was reported as caught, and the baseline, which ran in the original tree, could not show it; the giveaway was a mutant "caught" by a seventy-second gate in no time. *A work bound must cover every computation that precedes the kernel* (F86, F87, F90): all three were found by the first long run of a component that the gates had run only briefly. *Tests written with the code check what the code does* (F88), which is why the tools that make evidence have faults injected into them.

What the testing shows and does not show. The kernel has been compared with an earlier kernel and with an independently written checker on large numbers of generated and mutated inputs, attacked with blind generators on false statements, built with sanitizers, and mutated; every defect so found is recorded. None of this proves the kernel sound. The two checkers share one specification, and a defect of the specification is invisible to both; the generators sample distributions that the author chose; the mutants are the faults that the author imagined; and the lockstep comparison is blind to what the two kernels share. The statement of a theorem lies outside the chain altogether. Soundness is tested here and not proved, and we make no claim of being more trustworthy than any other kernel.

4 Using the system

ITP is used as a Python library. There is no surface language and no command-line prover: a definition, a tactic script and a query are Python, and a proof is a program that builds a term and submits it. This section shows the system in use. Each listing was run for this report (by `run_listings.py` of the supplementary material), and the output printed under it is the output of that run. The first two lines of every listing, which set up the module search path, are omitted here and are in the files.

4.1 Setting up

The kernel needs a C++23 compiler (it uses `std::expected`); the measurements of this report were made with g++ 13.3.0. Python 3.13 with `numpy`, `scipy` and `nanobind` runs the kernel, the libraries, the tactics and the gates; PyTorch and Gymnasium are needed only by the learner and by the Gymnasium wrapper of the environment, and Matplotlib only by the figures of the experiment report. From the root of the repository:

```
python3 -m venv .venv && source .venv/bin/activate
pip install "numpy>=2.5" "scipy>=1.18" "nanobind>=3.1"
make kernel         # builds the kernel and the frozen reference kernel
make gates-quick    # six kernel gates and the first example
```

On the machine of Section 5.2, building both kernels from scratch took 50 s and `make gates-quick` 4.5 s.

4.2 A theorem, a decision procedure, and a refusal

Listing 1 defines a function by structural recursion, proves a theorem about it by induction, closes a polynomial identity with `ring`, and then tries two things that must fail. A *world* is a kernel together with a name table; `nat_lib.build_nat()` returns one holding the library of unary naturals. `w.recursive` submits the function to the kernel, which checks that the recursion is structural; `w.prove` runs a tactic script over named hypotheses, extracts the finished term, and hands it to the kernel's checked entry point of Section 2.10, and only then does the theorem exist in the world. The false identity is refused by `ring` with the two normal forms in the message; the forged proof of $0 = 1$ never reaches the world, because the kernel checks it against the statement and finds that `Zero` and `Succ Zero` are different constructors. The independent checker then re-checks every global of the world, and the audit reports the axioms and quotient constants that a theorem depends on (here none).

Listing 1. A recursive function, an inductive proof, `ring`, and two refusals.

```
import itp_env_lnl as E                          # the compiled kernel
import nat_lib, recheck, audit
from author import AuthorError

w = nat_lib.build_nat()                          # a world: a kernel holding the library,
    ↪ every lemma admitted by it
Nat, Zero, Succ, Eq, forall = w['Nat'], w['Zero'], w['Succ'], w.Eq, w.forall_

# a recursive function (the kernel checks that the recursion is structural) ...
w.recursive('twice', w.arrow(Nat, Nat),
            lambda f: w.fun([Nat], lambda n: w.match(n, lambda _: Nat, [Zero, lambda k:
    ↪ Succ(Succ(f(k)))])))
twice = w['twice']

# ... and a theorem about it, proved by a tactic script
def by_induction(p):
    p.intro('n')
    p.induct('n', names=['k', 'ih'])             # two goals: Zero, and Succ k with the
    ↪ hypothesis ih
    p.refl()                                     # twice Zero = Zero + Zero, by computation
    p.rw('ih')
```

```
    p.rw('add_succ')                          # n + Succ m = Succ (n + m)
w.prove('twice_eq_add_self', forall([Nat], lambda n: Eq(twice(n), n + n)), by_induction)

# a polynomial identity, closed by `ring`
w.prove('binomial', forall([Nat, Nat], lambda a, b: Eq((a + b) ** 2, a ** 2 + 2 * a * b +
    ↪ b ** 2)),
        lambda p: (p.intro('a', 'b'), p.ring()))

# a false identity is refused, with both normal forms
try:
    w.prove('bad', forall([Nat, Nat], lambda a, b: Eq((a + b) ** 2, a ** 2 + b ** 2)),
            lambda p: (p.intro('a', 'b'), p.ring()))
except AuthorError as e:
    print('ring refuses:', str(e).splitlines()[0])

# the kernel does not trust the tactics: a forged proof of 0 = 1 is refused
try:
    w.k.declare_checked(Eq(Zero, Succ(Zero)).n, w['refl'](Nat, Zero).n, 'forged')
except (E.ITPError, RuntimeError) as e:
    print('kernel refuses:', str(e).splitlines()[0])

print('independent checker: %d globals re-checked, problems: %s' % (recheck.
    ↪ recheck_world(w)[0], recheck.recheck_world(w)[1]))
axioms, quotients = audit.dependencies_of_global(w.k, w.gid['binomial'])      # what the
    ↪ theorem rests on besides the kernel's rules
print('axioms:', sorted(axioms), '| quotient constants:', sorted(quotients))
```

```
ring refuses: ring failed: the two sides have different normal forms
kernel refuses: different constructors
independent checker: 132 globals re-checked, problems: []
axioms: [] | quotient constants: []
```

4.3 Integers, `linarith` and `nlinarith`

The integers are a library like any other: a quotient of pairs of binary naturals with every law proved (Section 3.2). In Listing 2, `ring` proves a polynomial identity over $\mathbb{Z}$; `linarith` finds non-negative multipliers by an exact simplex and the kernel checks that the weighted sum of the hypotheses and the negated goal is a negative constant, by `ring`; `nlinarith` adds products of hypotheses and squares and accepts a hint, here a proof of $0 \leq (x-y)^2$. A claim that does not follow is refused with the reason. Because the integers are built on the quotient, the audit of `am_gm` reports the five quotient constants and no axiom: the trusted base of a statement about integers includes the rule κ of Section 2.8, and this is visible.

Listing 2. Integers: `ring`, `linarith`, `nlinarith`, a refused claim, and an audit.

```
import int_lib, recheck, audit
from author import AuthorError

w = int_lib.build_int()                       # the integers: a quotient of pairs of
    ↪ binary naturals
Z, Eq, forall = w['Z'], w.Eq, w.forall_

w.prove('diff_squares', forall([Z, Z], lambda x, y: Eq((x + y) * (x - y), x * x - y * y))
    ↪ ,
        lambda p: (p.intro('x', 'y'), p.ring()))

# linarith: non-negative multipliers found by an exact simplex, the sum checked by `ring`
    ↪
w.prove('bound', forall([Z, Z, lambda x, y: x + y <= 10, lambda x, y, h: 3 <= x], lambda
    ↪ x, y, h1, h2: y <= 7),
        lambda p: (p.intro('x', 'y', 'h1', 'h2'), p.linarith()))

# nlinarith: products and squares, with a hint (a proof of 0 <= (x - y)^2)
def am_gm(p):
    p.intro('x', 'y')
    p.nlinarith(p.T('x y', lambda x, y: w['Z.sq_nonneg'](x - y)))
```

```
w.prove('am_gm', forall([Z, Z], lambda x, y: 2 * x * y <= x * x + y * y), am_gm)

try:                                          # a false claim is refused
    w.prove('false_claim', forall([Z, Z, lambda x, y: x <= y], lambda x, y, h: y <= x),
            lambda p: (p.intro('x', 'y', 'h'), p.linarith()))
except AuthorError as e:
    print('linarith refuses:', str(e).splitlines()[0])

n, problems = recheck.recheck_world(w)
print('independent checker: %d globals, problems: %s' % (n, problems))
axioms, quotients = audit.dependencies_of_global(w.k, w.gid['am_gm'])
print('axioms:', sorted(axioms), '| quotient constants:', sorted(quotients))
```

```
linarith refuses: linarith failed: no combination of the 2 facts read from the
    ↪ hypotheses refutes the negated goal
independent checker: 437 globals, problems: []
axioms: [] | quotient constants: ['Quot', 'Quot.ind', 'Quot.lift', 'Quot.mk', 'Quot.
    ↪ sound']
```

4.4 An episode in the environment

Listing 3 puts the same theorem to the environment instead of to a tactic script. An `Engine` serves one world; an `Episode` is one proof attempt on a statement, with the premises it may use. The menu lists the legal actions of the first open goal; an action that does not apply is refused and changes nothing; `INDUCT` is probed, taken, and the settling of the goals closes the base case by computation, leaving one goal with its induction hypothesis. The last lines run the blind search of the environment, iterative deepening over the menus with a budget of probes; the proof it returns has been replayed on a live proof and checked by the kernel, so a wrong search can only fail.

Listing 3. An episode: goal, menu, a refused action, an induction, and a blind search.

```
import collections
import nat_lib, gym_env as G, gym_search as S

w = nat_lib.build_nat()
Nat, Zero, Succ, Eq, forall = w['Nat'], w['Zero'], w['Succ'], w.Eq, w.forall_
w.recursive('twice', w.arrow(Nat, Nat),
            lambda f: w.fun([Nat], lambda n: w.match(n, lambda _: Nat, [Zero, lambda k:
    ↪ Succ(Succ(f(k)))])))
twice = w['twice']
statement = forall([Nat], lambda n: Eq(twice(n), n + n))

eng = G.Engine(w, kinds=set(G.Kind))                    # an engine that offers every
    ↪ kind of action
ep = G.Episode(eng, statement, premises=w.k.num_globals())  # one proof attempt; the
    ↪ agent may use every global so far
print(eng.render(ep.goal()))

menu = ep.menu()                                        # the legal actions for the
    ↪ first open goal
print(len(menu.actions), 'actions:', dict(collections.Counter(G.Kind(a.kind).name for a
    ↪ in menu.actions)))

try:                                                    # an action that does not
    ↪ apply is refused and changes nothing
    ep.step(G.Action(G.Kind.RING, 0))
except G.ActionError as e:
    print('refused:', e, '| goals open:', ep.n_goals, '| steps taken:', len(ep.steps))

induct = next(a for a in menu.actions if a.kind == G.Kind.INDUCT)
ep.step(induct)                                         # probed first, then taken;
    ↪ the base case is closed by computation
print(eng.render(ep.goal()))

proof, tree, searcher = S.prove_statement(eng, statement, max_depth=6, max_probes=2000)
    ↪   # blind iterative deepening over the menus
```

```
print('search:', 'found a proof' if proof is not None else 'no proof', '| probes used:',
    ↪ searcher.probes)
```

```
a0 : Nat
|- twice a0 = a0 + a0
17 actions: {'CASES': 1, 'INDUCT': 1, 'APPLY': 12, 'REWRITE': 2, 'RING': 1}
refused: the action RING(0) does not apply | goals open: 1 | steps taken: 0
h1 : Nat
h2 : twice h1 = h1 + h1
|- Succ (Succ (twice h1)) = Succ (h1 + Succ h1)
search: found a proof | probes used: 798
```

4.5 As a Gymnasium environment

Listing 4 loads the problem bank and plays random legal options on one problem. The observation is a dictionary of integer arrays (here only the shapes of three of them are printed), and every question the environment asks, here the kind of action, comes with a mask of its legal options. Random play over fifty decisions assembles four actions, of which the probe takes two and refuses two; a false statement could not be proved by any sequence of decisions, because the kernel checks the finished proof. Loading the bank builds a world of 748 globals in about 82 seconds and, with PyTorch loaded, 1.7 GB of memory (Section 5).

Listing 4. The problem bank as a Gymnasium environment; random play over legal options.

```
import random
import numpy as np
import bank_all, bank_env

bank = bank_all.load()                                    # the world of the bank:
    ↪ libraries, 268 statements, reference proofs
env = bank_env.BankEnv(bank, names=['two_pow_gt_n'], max_actions=40, max_decisions=400)
obs, info = env.reset(options={'problem': 'two_pow_gt_n'})
print({k: tuple(v.shape) for k, v in env.observation_space.spaces.items() if k in ('
    ↪ tokens', 'rows', 'mask')})
print('first question:', info['choice'].name, '|', int(info['action_mask'].sum()), '
    ↪ legal options')

rng = random.Random(0)
decisions = 0
while decisions < 50:
    legal = np.flatnonzero(info['action_mask'])           # the options of the current
    ↪ question that are legal
    obs, reward, terminated, truncated, info = env.step(int(rng.choice(legal)))
    decisions += 1
    if terminated or truncated: break
print('decisions:', info['decisions'], '| actions taken:', info['actions'], '| refused:',
    ↪ info['refused'], '| solved:', info['solved'])
```

```
{'mask': (1024,), 'rows': (1024, 4), 'tokens': (512,)}
first question: kind | 14 legal options
decisions: 50 | actions taken: 2 | refused: 2 | solved: False
```

4.6 Checking, auditing, and reproducing

Table 6 lists the commands that check the system itself and what each establishes. The independent checker and the audit of Listings 1 and 2 are also available on a whole world: `recheck_world` of the module `recheck` exports the kernel's environment and re-checks every global, and `SecondOpinion` of the same module builds the checker's environment from the globals that a given set of terms needs, in seconds instead of a minute, and refuses (fails closed) a term that mentions anything else. A reference proof is replayed and judged in the same way (gates S47, S50). The measurements of Section 5 are reproduced by the scripts of the `preprint/` directory of the artifact (Section 7); every number of the text is read from their output.

command	what it establishes	time
`make gates-quick`	the build works: six kernel gates and an example	4.5 s
`make gates`	the 49 default gates: kernel, tactics, environment, bank, contest proofs	45 min
`make gates-all`	all 54 gates, including the learner's, the repository's and the generator's	60 min
`ITP_VERIFY_MEMO=1 make gates`	the same, with every answer taken from a sharing table re-derived with the tables bypassed	about twice as long
`make mutants-check`	each of the 1,707 mutants and 252 tool faults still applies to the code it re-breaks	seconds
`python3 mutants/mutate.py` *prefix*	the gates fail on every mutant whose name begins with the prefix	minutes to hours
`make sanitize`	the gates of the default set that can run under AddressSanitizer and UndefinedBehaviorSanitizer (29 of 38)	305 s
`make dataset-verify`	every recorded proof and sub-theorem of the dataset replays in a fresh engine and is kernel-checked	25 min
`make audit`	the audit of the bank's statements against their intended meaning, with a sheet for second readers	minutes

Table 6. Commands that check the system and what they establish. Times are on the two-core machine of Section 5.2; those of the quick gates, of all the gates and of the sanitizer run were measured for this report, and the others are those given by the repository's documentation.

5 What the design buys, and how it differs

Section 3 describes the design; this section says what it is for, measures what it costs, and positions it among other systems. It contains no result of learning. Every claim about what a learner *gains* is a claim about the properties of the interface; whether a particular learner profits is an empirical question that this report does not answer.

5.1 What a learning loop needs, and what the design provides

A learning system uses a proof assistant in two ways: as a *judge* of finished proofs, whose verdict is the training signal, and as an *environment* that is called very many times. Table 7 lists six properties of the design that bear on these roles, the mechanism behind each, and what we measured at this commit (Section 5.2).

Why these properties matter to learning. In reinforcement learning on theorem proving the reward is sparse and comes from the judge: an episode that ends without a checked proof earns nothing. A judge that a policy could influence invites exploitation, and the kernel being the only way in (Section 2.10) removes the opportunity. Search and rollouts need states that can be restored, and a learner that can probe an action before it takes it can use the refusal as information; a probe costing milliseconds makes this affordable at the scale of a search. A finite menu, or a typed builder that offers only the tokens that keep a term well typed, removes a class of failures that a learner would otherwise have to learn to avoid: actions that do not parse. Budgets measured in kernel steps make a result a function of the task and not of the machine's load, which a comparison of two learners requires; and a budget that is reported as spent, and not as a refusal, keeps the learner from learning that an action is wrong when it was merely slow. Finally, a proof attempt that fails still contains theorems, and an environment that can extract them exactly turns a failure into data.

What the design costs. The calculus is smaller than that of Lean or Rocq (Section 6); the libraries hold 555 declarations, where the mathematical libraries of general-purpose assistants hold orders of magnitude more; the automation is limited to the decision procedures of Section 2.11 and the tactics

property	mechanism	measured (Section 5.2)
a verdict that the proposer cannot influence, cheap enough to be asked for every finished proof	one checked entry point (Section 2.10); a second checker as a witness (Section 3.3)	kernel check of a reference proof: median 21.5 ms, longest 6.43 s; the 516 declarations of the libraries in 0.417 s
trying an action and taking it back	checkpoint and rewind in time proportional to the allocation (Section 3.1); probes (Section 3.4)	median probe 0.607 ms, 99th percentile 26.4 ms; a checkpoint–rewind pair 308 ns
actions that are well formed by construction, and no text in the loop	menus; the typed term builder (Section 3.4)	one decision of the Gymnasium interface: median 0.421 ms, 1,513 decisions per second on one core
work measured in steps, not seconds	deterministic fuel; per-kind budgets; fail-closed exhaustion (Sections 3.1 and 3.4)	the outcomes of 8 tasks are identical in three runs (in process, one worker, two workers); 26 % of the 866 `APPLY` probes ended by the budget
scale-out by forking	one process holds the world; workers share it	two workers: speed-up 1.94 (efficiency 0.97), 2,080 MB proportional memory
failed attempts yield theorems	hindsight; portable actions; the recorder (Section 3.4)	exactness enforced by replay and kernel check (gate S47)

Table 7. Properties of the design that matter to a learning loop and the measurement behind each. The numbers are those of Section 5.2; none is a comparison with another system.

of Section 3.2; the interface is Python, and its cost is dominated not by the kernel but by the layer that assembles the questions (Section 5.2); and statements must be written in Python, since there is no surface language. A learner trained on text from other assistants cannot be pointed at this one.

5.2 Measurements

Setup. All timings are wall-clock measurements of the code the repository runs, on a virtual machine with 2 cores (Intel Xeon @ 2.10 GHz), 7.84 GB of memory, Linux 6.18.44, Python 3.13.16 and g++ 13.3.0, at commit f840710; one core is used unless stated. The measured scripts are in the artifact (Section 7) and every number in the text is read from their output. A *probe* is one try of an action on a goal by the engine, whoever asks, and is the unit in which the budgets of experiments are written; a *decision* is one answer to one question of the assembler of Section 3.4 (the kind of action, a lemma, a hypothesis, the next token of a term), so that an action takes several decisions and an action that carries a term takes many. Where a measurement was repeated (the library build, the re-check of the kernel, the micro-benchmarks) both runs are given as "A / B". The one-minute load average of the machine at the start of each run, which the output records, was at most 0.4 (on two cores).

Building and checking the libraries. The library chain of Table 2 is built from nothing in 3.82 s (3.83 s in a second run), with a peak resident memory of 69 MB. The values of its definitions and theorems have, counted as trees, 2,747,565 nodes, and the kernel holds 132,822 distinct nodes for the types and values together, a factor of 20.7 (Section 3.1); the largest value, `imo2019_p1_fwd`, has 341,553 nodes as a tree. Submitting each of the 516 checked declarations of the finished world to the kernel again, in an empty context, takes 0.417 s / 0.454 s, with a median of 59 µs per declaration, and the kernel rejects none. The independent checker takes 6.01 s / 5.10 s to load the world (export and parse) and 82 s / 76 s to check its 555 globals, about 182 times the kernel's time, and rejects none.

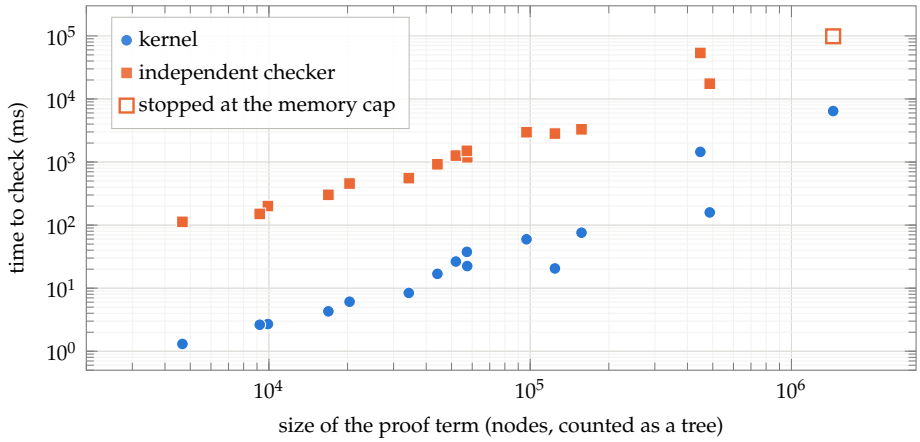


Figure 5. The cost of trusting a proof: time to check 16 reference proofs of the problem bank, sampled evenly by length, with the kernel and with the independent checker, against the size of the proof term. Both axes are logarithmic. The hollow marker is the time at which the independent checker stopped, at its memory cap, without a verdict.

operation	n	median (ms)	90th (ms)	99th (ms)
probe of an action	2,341	0.607	2.26	26.4
menu of a goal (mean 54 options)	543	4.11	20.1	30.5
start of an episode on a problem	40	0.952	4.10	6.41
replay of one recorded action	325	0.648	3.89	54.8
one decision of the Gymnasium interface	1,196	0.421	0.976	2.39

Table 8. Latency of the operations of the environment on the problem bank (milliseconds). Probes, menus, starts and replays are those of the blind search and a uniformly random policy on 8 problems; decisions are those of the instrumented Gymnasium loop with uniformly random legal choices.

The cost of trust on the problem bank. The kernel's check of a finished proof against its statement is cheap enough to be made for every proof an environment produces. Figure 5 plots, for 16 reference proofs of the bank sampled evenly by length, the time of the kernel's check and of the independent checker against the size of the proof term counted as a tree. The kernel's median is 21.5 ms and its longest check, of a term of 1,444,581 nodes, is 6.43 s. The independent checker, run in a child process with a cap of 120 s and 2,500 MB, accepted 15 of the 16 proofs, at a median of 1.19 s (longest 53.6 s) and between 37 and 138 times (median 57) the kernel's time, using a median of 17 MB (at most 1,257 MB); on the longest proof (`aime2002ii_p3`, 466 actions) it stopped at the memory cap after 99 s without a verdict, which is reported as such and not as a rejection. The independent checker is thus a witness, for the final proof and not for the inner loop; the table of every sampled proof is in Appendix B.

Latency of the environment. Table 8 gives percentiles of the operations of the environment, taken from the instrumented runs of the blind search and of a uniformly random policy on 8 problems of the bank that have reference proofs, evenly spaced in length; Appendix C gives the probes by kind of action. These are the costs of the engine on the actions that these two simple procedures try; a trained policy would try other actions, in other proportions. Appendix C shows that the budget does real work: of the 866 `APPLY` probes, 223 (26 %) ended with the budget spent and not with an answer, and the budget bounds what such a probe can cost whatever the goal and whoever asks. The slowest probe of the 2,341 took 1,319 ms (a `REFINE`, which supplies a term with holes); the slowest `APPLY` probe took 123 ms.

workload (one core, unless said)	unit	rate
blind search through the engine (8 problems)	probes per second	853
uniformly random choices through the Gymnasium interface	decisions per second	1,513
the same, driven for 10 s on each of the 8 problems	decisions / actions / probes per second	1,386 / 4 / 14
blind search in forked workers: one worker	probes per second	348
two workers	probes per second	677

Table 9. Throughput of the stack on the problem bank. The worker rows are of a different workload, the evaluation harness on 8 tasks with a budget of 100 probes each (808 probes in all), and are not comparable with the first; they measure scaling and not peak speed.

Throughput. Table 9 gives rates on one core. The engine, driven directly by the blind search, performs 853 probes per second (a range of 794 to 912 over two passes), and a problem takes 0.237 s of its budget of 200 probes; the search solves none of the 8 problems within that budget, and is a baseline for the cost of the machinery, not for difficulty. The time is divided as follows (an instrumented pass, which costs 1.17 times the plain one and changes no outcome): 59 % in the tactics that carry out the actions, 23 % in the rest of the probes (checkpoint, settling, rewind), 10 % in listing menus. Through the Gymnasium interface with uniformly random legal choices, the environment makes 1,513 decisions per second; 90 % of the time is spent in the assembler that builds the questions and the options and 6 % in the features, while probes, the tactics that carry them out, episode starts and replays together take 0.4 % and menus 1.1 % (a profiled pass of 1,196 decisions). A probe happens only when an action is complete, which under random choices takes 83 decisions on average in this pass, so the interface makes only 18 probes per second on this path; the same random policy driven for 10 seconds on each of the 8 problems makes 1,386 decisions, takes 4 actions and probes 14 times per second, 98 decisions to a probe, with 88 % of the time in the assembler. The cost of a decision is therefore the cost of the Python layer that assembles it, and not of the kernel or the engine; this is where a faster implementation of the interface would gain.

Scaling, memory, determinism. With the world built once and shared by forked workers, two workers on the two cores finish the same 8 tasks in 1.19 s against 2.32 s for one, a speed-up of 1.94 and an efficiency of 0.97. The memory of the pair, counted as proportional set size (what the processes cost together, where summing their resident sets would count the shared world again for each), is 2,080 MB; the world of the bank (748 globals, 268 problems) is built in 73 s and peaks at 1.3 GB (1.7 GB, and 82 s, in the run with PyTorch loaded); of the build, 11 % of the time (7.89 s) is spent in the kernel's checked entry points, which received 681 declarations, and the rest in the Python layer that runs the tactics which construct the proofs. Nothing is claimed beyond two workers: the machine has two cores. The result rows of the three runs (in process, one worker, two workers) agree on every field that is not a measurement of time, for all 8 tasks and with none left out for coming near the time limit; their outcome digest is `e0640db88b137869`, the same on any machine that finds the same proofs after the same number of probes. The world of the libraries has the same digest of the names and types of its globals, `cc4eb4af3abcc541...`, in both runs of the build.

Micro-benchmarks. A call across the Python–C++ boundary costs 49 ns / 38 ns; a checkpoint followed by a rewind of an empty region 308 ns / 147 ns; building a node, 753 ns / 613 ns from Python; and rewinding after about 6,000 nodes were allocated 21 µs / 15 µs, or 3.5 ns / 2.6 ns per node. Binary numerals are evaluated by reduction in the kernel: a product of a 317-bit and a 337-bit numeral is decided in 58 ms / 44 ms, and one of 793 by 843 bits in 409 ms / 276 ms.

What the measurements do not show. They are of one machine with two virtual cores, and a larger machine would change the absolute numbers and, above two workers, the scaling. The workloads are a blind search and a random policy, and the latency of a trained policy's actions was not measured.

The sample of reference proofs for the cost of trust is 16 of 187. No figure here is compared with a figure for another system.

5.3 How it differs from other systems

We contrast design choices and give no speed comparison: the published figures of other systems are for other tasks on other hardware, and a fair comparison would formalise the same statements in each system and time both on one machine, which we have not done.

Learners attached to general-purpose assistants. CoqGym and GamePad (Coq), HOList (HOL Light), lean-gym, LeanDojo, Pantograph and the Kimina Lean Server (Lean) connect a learner to an existing proof assistant that runs as a separate process, and exchange text or serialised terms with it [4, 5, 20, 31, 45, 60, 61]. What this gives a learner that ITP cannot give is the assistant's libraries of formalised mathematics, its automation, its community, and a kernel with a long record. Much of the engineering of these systems goes into making the interface dependable; LeanDojo, Pantograph and the Kimina server are in part that. ITP draws the boundary elsewhere: the environment is in the process of the kernel, a state is a value that can be checkpointed, and an action is data and not syntax. The price is the one listed in Section 5.1.

Purpose-built systems. INT, an environment and a generator of problems for inequalities, Peano, a dependently typed environment with a finite action space, and Minimo, which conjectures and proves in Peano, are environments built for learning and not interfaces to an existing assistant [47, 48, 57]. ITP shares the aim and is closest to Peano in using dependent types and a finite menu of actions. Its own choices are a calculus with inductive families, quotients and proof irrelevance (Section 2), a deterministic work budget in the kernel, a second checker and a mutation catalogue (Section 3.5), a typed term builder with which an agent says arbitrary well-typed terms (Section 3.4), and a bank of competition problems with a family-disjoint split. Hindsight relabelling on proof search trees is used by Minimo, and the idea is hindsight experience replay [3]; we use it for the same purpose and claim nothing for it.

Trust. The kernel of Lean is small too; its type theory has been analysed formally [10], and an independent implementation of its kernel exists [11]. We do not claim a more trustworthy kernel. The claim is narrower: the whole stack up to the action interface is small enough to be tested as a unit, by mutation among other means; the cost of every call is a deterministic budget; and a second implementation of the specification is part of the repository (Section 3.5).

Learners that use text. Language-model provers [28, 33, 37–39, 49–51, 58, 59] and benchmarks such as miniF2F [62] work on the surface syntax of Lean, Metamath or Isabelle. ITP has no surface language, so these models and corpora do not transfer to it; its interface is aimed at policies that read tokens of goals and choose among options. A problem bank of 268 statements is small beside these benchmarks; what it offers is five levels of difficulty, a tier of statements without proofs that a prover is asked to find, and splits that keep the problems of a family together.

6 Limitations

This section collects what the system does not do and what the evidence does not show. It is organised by layer. Where the log of findings (Table 5) has an entry, its number is given. The log holds 91 findings; 4 are open: F28 (conversion), F61 (the assembler of the policy interface), F67 (the time of the independent checker), and F70, which records a measurement of an early learner on an earlier version of the bank and not a defect of the system.

6.1 The calculus

Universes. There is no cumulativity and there is no universe polymorphism (Section 2.2). A definition used at two levels is written twice, and a term of one level is used at another by an explicit construction. The eleventh kind of node, meant for level parameters, is not used.

Inductive types and recursion. A declaration is of one family (A6 of Section 2.5): there are no mutual families, no occurrence of the family nested in an argument of another (`List Bad`), no coinductive types, and no inductive–recursive or inductive–inductive definitions. Recursion is structural on one argument of one family, with no mutual recursion; well-founded recursion is not a primitive. Pattern matching on an indexed family is the kernel's `match` with index forcing (Section 2.5), which is first-order: it solves a field that stands alone at an index position, compares a field met twice, matches equal constructors, refutes two different ones, and declines everything else; equations between indices that need computation to solve are left to the user, and a refuted branch must still be written. The argument that the forcing is sound is not formalised, and it is the most delicate part of the kernel. Discrimination and injectivity lemmas are generated for families without indices or parameters only, there is no heterogeneous equality, and `cases` does not use them by itself (F2b).

Conversion. Conversion has η for functions and for nothing else; there are no records, and no η for the unit type. Definitional proof irrelevance holds only in the Prop configuration, and the calculus has uniqueness of identity proofs there, so it is not a setting for univalent development. The decision procedure is incomplete and not transitive on stuck case analyses (F28): two terms that each convert to their own normal form can fail to convert to each other, so that `exact` or `apply` can miss an equality that normalising both sides exposes. The failure is in the safe direction, since a positive answer is always justified by evaluation. Conversion is exponential in the depth of a shared term whose two sides unfold differently (F29); the work bound turns the blow-up into an exhausted budget, and the root cause is open. A fixpoint is unfolded at any argument and not only at a constructor (F23), which is mitigated for the proofs that the tactic layer builds and remains a difference from the specification (Section 2.4).

Axioms, language, libraries. One axiom is shipped, excluded middle; the quotient rule is part of the kernel, and the integers depend on it (Section 2.8). The audit of a theorem lists the axioms and quotient constants it depends on. There is no surface language: a statement is a Python expression over the kernel's term builder, and there are no implicit arguments or coercions for the author, no type classes, no notation mechanism beyond infix names, and no module system. The libraries (555 declarations, Table 2) cover the naturals, integers and some number theory; there are no real numbers, no algebraic structures, and no geometry or combinatorics.

Metatheory. Subject reduction, strong normalisation and consistency are not proved for the calculus as implemented, and no part of its metatheory is mechanised (Section 2.12). The two deviations of Appendix A are places where the implementation is weaker than the specification it was written against. The second can be exploited: in a world with excluded middle and propositional extensionality as axioms, a family that the kernel admits yields a proof of False, which the kernel and the independent checker both accept (Listing 7). We know of no exploit of the first, and we have not proved that there is none. Neither deviation is repaired in the version of the repository that holds this report. Neither is reachable from the environment, whose actions can neither declare a family nor say a `match`; both are reachable from the Python interface of the kernel, which is what a library author uses, and the exploit also needs an axiom that no shipped library assumes.

6.2 Trust and testing

Soundness is tested, not proved. The trusted base is the kernel (7,500 lines of header and 788 of bindings) and the entry points that guard it, the C++ compiler and the Python interpreter that call it.

The evidence that it is sound is the apparatus of Section 3.5, which finds defects and cannot show their absence. Its second checker (848 lines) is a re-implementation from the same specification by the same author, so it does not catch a flaw in the specification or a misreading that both share, and it does not re-check the admission of inductive families (D2 of Appendix A), which is why it accepts the proof of False of Listing 7. The mutants are written by the author of the code and measure the tests against the faults that the author can imagine, and the 22 equivalent mutants are the author's judgement.

The independent checker is slow. It takes about 182 times the kernel's time over the libraries, between 37 and 138 times on the sample of reference proofs, and it needed more than 2,500 MB on one proof, where it stopped without a verdict. The log records case-enumeration proofs for which it takes minutes (F67), which the gates check with the independent checker only on request. It is a witness for a finished proof, not a component of a loop.

What was and was not run for this report. The gates, the sanitizer run, the library and environment measurements, and the listings were run at the commit of the report on one machine: Linux, x86-64, two virtual cores, g++ 13.3.0. The three listings of Appendix A.2 that prove False and check the proof in Lean were added afterwards and run on a kernel with the same digest; the two in Lean were run with Lean 4.15.0. The campaign of falsification by fuzzing (`make soundness`), a single pass of the whole mutation catalogue against the final commit, the gates run with the sharing tables of the kernel bypassed (`ITP_VERIFY_MEMO=1`), the long gates of the bank under the sanitizers, the leak detector of the sanitizer, and other platforms and compilers were not run; the repository contains the commands for each. The measurements of cost are of this machine. The counts and digests are properties of the code, and the times are not.

Budgets. The work bound of the kernel counts evaluation steps, and the tactics that compute in Python before the kernel sees anything have bounds of their own. Three places where a bound was missing were found by long runs and not by any gate (F86, F87, F90), and we know of no argument that there are no others.

6.3 The environment and the bank

The interface. An episode acts on the first open goal; there is no choice of goal and no tactic combinator, and a proof is a sequence of single actions. The automation is the decision procedures for polynomial arithmetic over the naturals and the integers (`ring`, `linarith`, `nlinarith`, `linear_combination`), enumeration over an interval, and rewriting; there is no general-purpose simplifier with a lemma database that grows, no SMT back end, and no tactic for the arithmetic of the reals. A term with `match` or `let` cannot be said to the typed builder, which concerns two proofs of the libraries and the statement of functional extensionality, and a lemma above the premise limit cannot be said. The assembler, the policy-facing interface, says 8,789 of the 9,248 actions of the 648 recorded proofs of the dataset (the reference proofs of the bank in both regimes, and the libraries' proofs), at 1.08 ms an action. It cannot say the other 459: 424 in the reference proofs of the bank in the regime that offers every earlier global, 8 in the second proofs of the strict regime, and 27 in the libraries. They are of the kinds `FORWARD` (183), `CLAIM` (155), `REFINE` (60), `USE` (46), `APPLY_WITH` (12), `LINCOMB` (3), and in each of them some token that the action needs is not among the options offered at that point. The log of findings (F61) attributes the failures that it had counted to steps whose terms fit their holes only up to computation, which the matcher of the typed builder does not see through; that entry was written at an earlier state of the repository and its figures do not match these, and we have not classified the present failures further. A policy trained on the recorded actions would not meet these, and a measure of how often it can reproduce a recorded proof is bounded by the fraction that the assembler can say. The recorder replays 453 of the 459 recorded proofs of the libraries, and the six that it does not are named in a gate. In the prefix regime a problem with a helper on the menu can be a single action, which measures the use of a lemma and not the finding of

level	with a reference proof	statement only	all
1	37	0	37
2	67	0	67
3	68	7	75
4	14	31	45
5	1	43	44
all	187	81	268

Table 10. The problems of the bank by level (how much thinking a problem asks for, not how long its proof is) and by whether a kernel-checked reference proof exists. The counts are read from the released dataset file.

an idea; the strict regime hides the helpers, and only the problems with a self-contained reference proof have one.

The bank. It is small (268 problems) and tilted toward the easy end. Table 10 gives the levels: 172 of the 187 problems with a reference proof are of levels 1 to 3, and of the 44 problems of level 5 one has a proof. Of the 60 problems from the IMO, its shortlist and the APMO, 12 have a proof; the others are statements, with exhaustive evidence on small ranges that they are true. Of the problems, 72 are formalised as in a contest, 19 are changed to fit the vocabulary (integers instead of reals, one part of several), 72 are written for the bank in the manner of a contest and are not contest problems, and 105 are classical facts and exercises; the texts of the AIME problems are paraphrased. The topics are arithmetic, algebra and number theory (163 problems carry the number-theory tag); 0 are tagged combinatorics and none is geometry. That a statement says what its problem says is checked by evaluation on windows of small values against an executable reading written by the person who formalised it, and no second reader has recorded a verdict; a misreading shared by the statement and its executable reading is invisible to every check. The split is by family, so that the test problems are not restatements of training problems, but 58 problems in a test split, eight of them statements without proofs, are a small test.

Hindsight and throughput. Hindsight returns the sub-theorems of a proof tree that can be made exact; those that cannot are dropped and counted, so the yield on a given attempt is less than the number of its solved sub-goals. The interface is in Python, and a decision costs the cost of the assembler (Section 5.2); the throughput of a decision is therefore far from the throughput of the kernel. Scaling was measured to two workers on a machine with two cores. No GPU or accelerator is involved in the environment, and none was measured.

No learning. The report contains no experiment in which a policy was trained on the environment, and claims nothing about what a learner would achieve with it. Section 5 describes properties of the interface and what they cost; whether they help a learner is the subject of future work, and the comparison with other systems is of design and not of speed.

7 Conclusion

We have described ITP, a proof assistant built to be the environment of a learning system: a dependent type theory with inductive families, an impredicative bottom universe, quotients and a structural guard, decided by a C++ kernel of about 7,500 lines; a Miller-pattern unifier and certificate-checked decision procedures for polynomial arithmetic; a second, independent checker; and a layer that turns the assistant into an environment, with a finite menu of actions and a typed term builder for the rest, probes that try an action and take it back, work counted in kernel steps, exact hindsight, a bank of 268 problems, and a Gymnasium interface with no text in the loop.

What the evidence supports is modest and concrete. A probe of an action costs a median of 0.607 ms and a decision of the Gymnasium interface 0.421 ms on one core of a two-core machine;

the kernel checks the median reference proof of the bank in 21.5 ms, so that the verdict can be had for every finished proof, and the independent checker confirms it about 57 times more slowly, as a witness. The soundness of the kernel is tested, not proved, by a frozen reference kernel run in lockstep, a second checker, canaries, a fuzzer, 1,707 mutants, sanitizers and a log of 91 findings. The testing also reached its own limits: writing the rules down for this report turned up two places where the kernel checks less than the usual conditions on a motive and on the positivity of a family (Appendix A); the second checker, written from the same specification, shares the first omission in part and does not examine the second. The second omission can be exploited: with excluded middle and propositional extensionality added to a world, a family that the kernel admits gives a proof of False that both checkers accept (Appendix A.2).

What the evidence does not show is whether a learner profits. The report has no learning experiment, and its claims about the design are claims about an interface: a verdict the proposer cannot influence, states that can be restored, actions that are well formed by construction, a cost that is a function of the task, and failures that yield theorems. Whether these properties help is an empirical question that needs a fixed protocol, an untrained control, and a budget measured in the unit that the environment provides; the environment is meant to make that experiment cheap to run and its outcome checkable.

Future work. The two deviations of Appendix A are not repaired in this version and should be, in the kernel, with a gate for each (for the second, the proof of False of Listing 7 must fail at the declaration of `R`), and the independent checker should re-check the admission of inductive families. The remaining gaps of the calculus, universe polymorphism, mutual and nested families and a real-number library, bound the mathematics that the bank can contain; the bank itself needs a second reader of its statements and proofs for its hard tier. A faster implementation of the policy interface would move the throughput of a decision toward that of the kernel. Part of the metatheory, starting with the guard and the forcing of indices, could be mechanised. And the environment should be used: to train and to compare learners on the bank and on synthetic theorems, against an untrained control, with the budgets of this report.

Availability and reproducibility

The code, the dataset of the problem bank, the sources of this report and the scripts that produced its numbers are in one repository at commit f840710 (`https://github.com/Aditya240307e/itp-kernel`); a snapshot is archived with the report (DOI 10.5281/zenodo.24047732). The directory `preprint/` of the repository holds the sources of the report (`src/`), the bibliography, the measurement scripts, the listings and their outputs, and the raw measurements as JSON and logs; `python3 -I preprint/build.py` regenerates every macro that holds a number, and then the PDF, from them. The scripts and their costs on the machine of Section 5.2 are: `measure.py`, the library chain, its re-check by the kernel and by the independent checker, and the micro-benchmarks (about three minutes); `measure_check.py`, the cost of trusting a sample of reference proofs (about ten minutes, 2 GB); `src/python/experiments/exp_bench.py`, the latency and the throughput of the environment (a few minutes after the world is built); `measure_assembler.py`, the round trip of the policy interface; `measure_gates.py`, the gates named in Table 4 and the sanitizer run; and `run_listings.py`, the listings of Section 4 and Appendix A (those in Lean need a Lean 4). The code, the dataset and the world are identified by digests so that a rerun can be compared with ours: the digest of the directory of the kernel's sources (`src/c++/hybrid`) is `7b17122355090fbe`, that of the names and types of the 555 globals of the library world is `cc4eb4af3abcc541...`, and that of the outcomes of the benchmark tasks is `e0640db88b137869`. The counts of this report (lines, declarations, mutants, findings, problems, proofs) are read from the files of the repository and do not depend on the machine; the times do.

A Two deviations of the kernel

The two deviations below were found while the rules of Section 2 were written down from the code, by reading it and by probing it; no gate found either. The second checker shares part of the first omission (it does not compare the binder types of a motive either, though it does require the motive to end in a sort) and does not examine the admission of a family at all (Section 3.3). The second deviation can be exploited: once excluded middle and propositional extensionality are added to a world, a family that the kernel accepts yields a proof of False, which the kernel and the second checker both accept (Appendix A.2). Neither deviation is repaired in the version of the repository that holds this report: the sources of its kernel are those of commit f840710, the commit of the measurements (digest `7b17122355090fbe`). Each deviation has a listing in the supplementary material that reproduces it (`preprint/listings/dev_motive.py` and `dev_positivity.py`), and the exploit has three more, one in the system and two in Lean (`dev_exploit.py`, `dev_lean_decl.lean` and `dev_lean_false.lean`); Listings 5 to 9 show them with the output of a run on that kernel.

A.1 D1: the motive of a match is checked for its shape

The usual rule for match s return M with $\overline{b}$ with $s : I\,\overline{a}$ requires $M : \Pi(\overline{y}{:}\overline{Y})(z{:}I\,\overline{y}).\,S$, where $\overline{Y}$ are the index types of I at the parameters of s and S is a sort. The kernel's check of an explicit motive (`verify_motive_shape`) infers the type of M, evaluates it, and checks that it is a chain of $|\overline{a}|+1$ products. It does not compare the binder types with $\overline{Y}$ and $I\,\overline{y}$, and it does not require that what follows the last product is a sort (the message of its error speaks of a chain "ending in a sort", but the code reads the level of the sort only when the value is one, and otherwise goes on).

Listing 5. Deviation D1: three motives, two of which the kernel does not refuse.

```
import nat_lib, recheck

w = nat_lib.build_nat()
Nat, Zero, False_ = w['Nat'], w['Zero'], w['False']
B2 = w.inductive('B2', w.TYPE, [('b2f', []), ('b2t', [])])          # a second data type,
    ↪ unrelated to Nat

def attempt(label, make, name=None):
    try: t = make(); ty = w.type_of(t)
    except Exception as e: print('%s: refused (%s)' % (label, str(e).split('\n')[0]));
    ↪ return
    print('%s: the kernel infers %s' % (label, w.show(ty)))
    if name:                                                          # the kernel's
    ↪ checked entry point
        try: w.k.declare_checked(ty.n, t.n, name); print('    declared: accepted')
        except Exception as e: print('    declared: refused (%s)' % str(e).split('\n')
    ↪ [0])

# (a) match on a Nat with a motive whose binder is a B2
attempt('(a) motive over B2', lambda: w.fun([Nat], lambda n: w.match(n, w.fun([B2],
    ↪ lambda z: Nat), [Zero, lambda k: Zero])), 'probe_a')
# (b) a motive whose codomain is a term (Zero) and not a sort
attempt('(b) motive into Zero', lambda: w.fun([False_], lambda e: w.match(e, w.fun([
    ↪ False_], lambda z: Zero), [])), 'probe_b')
# (c) a motive without a binder
attempt('(c) motive Nat', lambda: w.fun([Nat], lambda n: w.match(n, Nat, [Zero, lambda k:
    ↪  Zero])))
n, bad = recheck.recheck_world(w)
print('independent checker: %d globals re-checked, problems: %s' % (n, bad))
```

```
(a) motive over B2: the kernel infers Nat -> Nat
    declared: accepted
(b) motive into Zero: the kernel infers False -> Zero
    declared: refused (Pi codomain must be a type)
(c) motive Nat: refused (ill-typed term: explicit motive_id is not shaped like a valid
    ↪ motive -- expected a Pi chain of (index count + 1) arguments, ending in a sort)
independent checker: 133 globals re-checked, problems: []
```

In case (a) the motive abstracts over a second data type and is applied, in the types of the branches, to terms of `Nat`; the kernel infers a type, the checked entry point accepts the declaration, and the independent checker accepts it too: it checks the number of products and that the codomain is a sort, and does not compare the binder types either. In case (b) the codomain of the motive is a term and not a sort; the kernel infers a type for the match, and the checked entry point refuses the declaration because that type is not a type. Nothing of (b) remains for the independent checker to re-check, and its source states the condition on the sort that the kernel's shape check lacks. Case (c), a motive with no binder, is refused.

Exposure. The function form of `match` of the authoring layer builds the motive's binder from the type of the scrutinee, and cannot produce (a); an indexed family needs the explicit form, and so does any code that builds terms directly. The actions of the environment cannot say a `match` (Section 3.4). We have not scanned the matches of the shipped terms for ill-formed motives, and we do not know whether (a) can be exploited to derive a contradiction. A motive that analyses its argument would be applied to terms of the wrong type, and the untyped evaluator would reduce such an application by the shape of the term.

Repair. Compare each binder type with the expected one by conversion, and require the codomain to be a sort, in `verify_motive_shape`; add the cases of the listing to the gates.

A.2 D2: strict positivity does not look in the indices

Condition A5 of Section 2.5 examines the types of the fields: a field type is accepted if the family does not occur in it, or if it is a product whose domain does not contain the family and whose codomain is accepted, or if its head is the family. Nothing is examined inside the arguments of that head, and nothing inside the result indices of the constructor. The usual conditions for indexed families exclude the family from both [21, 46].

Listing 6. Deviation D2: a negative occurrence in the index of a recursive field, and in a result index.

```
import nat_lib, recheck

w = nat_lib.build_nat()
TYPE, PROP, False_ = w.TYPE, w.PROP, w['False']

def attempt(label, make):
    try: make(); print('%s: accepted' % label)
    except Exception as e: print('%s: refused (%s)' % (label, str(e).split('rejected: ')
    ↪ [-1]))

# control: the family on the left of an arrow in a field is refused
attempt('B0, field (B0 -> False) -> B0', lambda: w.inductive('B0', TYPE, [('mk0', [
    ↪ lambda *_: w.arrow(w.arrow(w['B0'], False_), w['B0'])])]))
# the family inside the index of a recursive field
attempt('Q A, field Q (Q A -> False)', lambda: w.inductive('Q', w.arrow(PROP, TYPE), [('
    ↪ mkq', [PROP, lambda A: w['Q'](w.arrow(w['Q'](A), False_))], lambda A, h: [A])]))
# the family inside a result index of a constructor
attempt('R A, result index R A -> False', lambda: w.inductive('R', w.arrow(PROP, TYPE),
    ↪ [('mkr', [PROP], lambda A: [w.arrow(w['R'](A), False_)])]))
n, bad = recheck.recheck_world(w)
print('independent checker: %d globals re-checked, problems: %s' % (n, bad))
```

```
B0, field (B0 -> False) -> B0: refused (itp: constructor 'mk0' fails strict positivity --
    ↪  the inductive being defined appears to the left of an arrow in an argument type)
Q A, field Q (Q A -> False): accepted
R A, result index R A -> False: accepted
independent checker: 134 globals re-checked, problems: []
```

The family `Q` of Listing 6 has a field of type $\mathsf{Q}\,(\mathsf{Q}\,A \to \mathsf{False})$, in which Q occurs to the left of an arrow inside an index; `R` has $\mathsf{R}\,A \to \mathsf{False}$ as a result index. Both are accepted; the control, with the family to the left of an arrow in a field, is refused. The independent checker re-checks the declarations

of the constants and finds nothing, because it does not re-check the admission of inductive families (Section 3.3).

An exploit. The family `R` yields a proof of False once two standard classical axioms are added to the world: excluded middle, $\forall P$:Prop. $P \vee \neg P$, and propositional extensionality, $\forall P\, Q$:Prop. $(P \leftrightarrow Q) \rightarrow P = Q$. Write $E\, x$ for $\mathsf{R}\, x \rightarrow$ False. The constructor gives $\mathsf{R}\,(E\, B)$ for every proposition B, and, since it is the only constructor, the elimination of an inhabitant of $\mathsf{R}\, x$ into a proposition gives $\exists B.\ E\, B = x$ (`R_inv`). So $\mathsf{R}\, x$ is inhabited exactly when x is a value of E; hence $E\, x$ is refuted when x is a value of E (`K1`) and holds when it is not (`K3`), and neither lemma uses an axiom. With the two axioms every proposition is equal to True or to False, so E has at most two values, and each of the cases for E False and E True contradicts "$E\, x$ holds exactly when x is not a value of E". Listing 7 carries this out through the checked entry point of the kernel, in the base library of the repository extended by the two axioms and the family; it ends with `boom : False`. The audit of `boom`, the counterpart of the `#print axioms` of Lean, lists the two axioms and no others, and the independent checker re-checks every declaration of the world, `boom` included, and finds no problem, since it does not look at the admission of `R`.

Listing 7. Deviation D2 exploited: a proof of `False` from `R`, excluded middle and propositional extensionality.

```
import lnl_base, audit, recheck

w = lnl_base.build_base()                 # the base library of the repository
PROP, TYPE, F, T = w.PROP, w.TYPE, w['False'], w['True']
fa, Eq, Ex, app = w.forall_, w.Eq, w.Ex, w.app
# two standard classical axioms: excluded middle and propositional extensionality
w.axiom('LEM', fa([PROP], lambda P: P | ~P))
w.axiom('propext', fa([PROP, PROP, lambda a, b: w.Iff(a, b)], lambda a, b, h: Eq(a, b)))
# the family R of dev_positivity.py, which the kernel accepts
w.inductive('R', w.arrow(PROP, TYPE), [('mkr', [PROP], lambda A: [w.arrow(w['R'](A), F)])
    ↪ ])
R = w['R']
E = lambda x: w.arrow(R(x), F)            # E x := (R x -> False)
print('R   : %s\nmkr : %s' % (w.show(w.global_type('R')), w.show(w.global_type('mkr'))))

def prove(name, stmt, script):            # the kernel's checked entry point
    w.prove(name, stmt, script)
    print('%-9s %s' % (name, w.show(w.global_type(name))))

# an element of R x comes from some B with E B = x ...
prove('R_inv', fa([PROP, lambda x: R(x)], lambda x, r: Ex(PROP, lambda B: Eq(E(B), x))),
      lambda p: (p.intro('x', 'r'), p.cases('r', names=['A']), p.exists('A'), p.refl()))
# ... and conversely
prove('R_of_eq', fa([PROP, PROP, lambda B, x: Eq(E(B), x)], lambda B, x, h: R(x)),
      lambda p: (p.intro('B', 'x', 'h'), p.cases('h'), p.apply('mkr')))
# K1: if x is a value of E, then E x is refuted
prove('K1', fa([PROP, PROP, lambda B, x: Eq(E(B), x), lambda B, x, h: E(x)],
               lambda B, x, h, e: F),
      lambda p: (p.intro('B', 'x', 'h', 'e'), p.apply('e'),
                 p.exact(('R_of_eq', 'B', 'x', 'h'))))
# K3: if x is not a value of E, then E x holds
prove('K3', fa([PROP, lambda x: w.arrow(Ex(PROP, lambda B: Eq(E(B), x)), F)],
               lambda x, hn: E(x)),
      lambda p: (p.intro('x', 'hn', 'r'), p.apply('hn'), p.exact(('R_inv', 'x', 'r'))))

# small facts about Prop (the last two use propext)
tr_ = lambda a, b, c, h1, h2: app(w['trans'], PROP, a, b, c, h1, h2)
prove('tf', fa([Eq(T, F)], lambda h: F),
      lambda p: (p.intro('h'),
                 p.exact(lambda p: app(w['transport'], PROP, w.lam(PROP, lambda P: P),
                                       T, F, p.h('h'), w['tt']))))
prove('H4', fa([PROP, lambda X: Eq(X, F), lambda X, a: Eq(X, T)], lambda X, a, b: F),
      lambda p: (p.intro('X', 'a', 'b'), p.apply('tf'),
                 p.exact(lambda p: tr_(T, p.h('X'), F,
                                       app(w['symm'], PROP, p.h('X'), T, p.h('b')),
                                       p.h('a')))))
```

```
prove('eq_true', fa([PROP, lambda P: P], lambda P, h: Eq(P, T)),
      lambda p: (p.intro('P', 'h'), p.apply('propext'), p.apply('conj'),
                 p.intro('x'), p.apply('tt'), p.intro('x'), p.exact('h')))
prove('eq_false', fa([PROP, lambda P: ~P], lambda P, n: Eq(P, F)),
      lambda p: (p.intro('P', 'n'), p.apply('propext'), p.apply('conj'),
                 p.intro('x'), p.apply('n'), p.exact('x'), p.intro('x'), p.cases('x')))

Efun = w.lam(PROP, lambda P: E(P))
# from e : B = v, the equation E B = E v
congr = lambda p, v, e: app(w['f_equal'], PROP, PROP, Efun, p.h('B'), v, p.h(e))

def boom(p):
    # excluded middle and propext make E False and E True each equal to True or to False
    p.have('c0', ('LEM', E(F)))
    p.cases('c0', names=['h0', 'h0'])
    # E False holds, so E False = True: True is a value of E, so E True is refuted,
    # so E True = False: False is a value of E, so E False is refuted
    p.have('q0', ('eq_true', E(F), 'h0'))
    p.have('n1', ('K1', F, T, 'q0'))
    p.have('q1', ('eq_false', E(T), 'n1'))
    p.have('n0', ('K1', T, F, 'q1'))
    p.exact(('n0', 'h0'))
    # E False fails, so E False = False
    p.have('q0', ('eq_false', E(F), 'h0'))
    p.have('c1', ('LEM', E(T)))
    p.cases('c1', names=['h1', 'h1'])
    # E True holds, so E True = True: True is a value of E, so E True is refuted
    p.have('q1', ('eq_true', E(T), 'h1'))
    p.have('n1', ('K1', T, T, 'q1'))
    p.exact(('n1', 'h1'))
    # E True fails too: both values of E are False, so True is not a value of E,
    # so E True holds
    p.have('q1', ('eq_false', E(T), 'h1'))
    def not_in_image(p):
        p.intro('hex'); p.cases('hex', names=['B', 'hB'])
        p.have('cB', ('LEM', 'B')); p.cases('cB', names=['b', 'b'])
        # B = True, so E B = E True = False, but E B = True
        p.have('e1', ('eq_true', 'B', 'b'))
        p.have('e2', lambda p: congr(p, T, 'e1'))
        p.have('e3', lambda p: tr_(E(p.h('B')), E(T), F, p.h('e2'), p.h('q1')))
        p.exact(lambda p: app(w['H4'], E(p.h('B')), p.h('e3'), p.h('hB')))
        # B = False, so E B = E False = False, but E B = True
        p.have('e1', ('eq_false', 'B', 'b'))
        p.have('e2', lambda p: congr(p, F, 'e1'))
        p.have('e3', lambda p: tr_(E(p.h('B')), E(F), F, p.h('e2'), p.h('q0')))
        p.exact(lambda p: app(w['H4'], E(p.h('B')), p.h('e3'), p.h('hB')))
    p.have('ni', w.arrow(Ex(PROP, lambda B: Eq(E(B), T)), F), not_in_image)
    p.exact(lambda p: app(p.h('h1'), app(w['K3'], T, p.h('ni'))))

prove('boom', F, boom)

# what the proof rests on (checker/audit.py, the counterpart of Lean's #print axioms)
for name in ('K1', 'K3', 'boom'):
    axioms, quotients = audit.dependencies_of_global(w.k, w.gid[name])
    print('axioms of %-5s %s' % (name + ':', ', '.join(sorted(axioms)) or 'none'))
# the independent checker re-checks every declaration of this world, boom included
n, bad = recheck.recheck_world(w)
print('independent checker: %d globals re-checked, problems: %s' % (n, bad))
```

```
R   : Prop -> Type
mkr : forall (x0 : Prop), R (R x0 -> False)
R_inv     forall (x0 : Prop), R x0 -> Ex Prop (fun (x2 : Prop) => (R x2 -> False) = x0)
R_of_eq   forall (x0 : Prop), forall (x1 : Prop), (R x0 -> False) = x1 -> R x1
K1        forall (x0 : Prop), forall (x1 : Prop), (R x0 -> False) = x1 -> (R x1 -> False)
    ↪  -> False
K3        forall (x0 : Prop), (Ex Prop (fun (x1 : Prop) => (R x1 -> False) = x0) ->
    ↪ False) -> R x0 -> False
tf        True = False -> False
H4        forall (x0 : Prop), x0 = False -> x0 = True -> False
```

```
eq_true   forall (x0 : Prop), x0 -> x0 = True
eq_false  forall (x0 : Prop), Not x0 -> x0 = False
boom      False
axioms of K1:   none
axioms of K3:   none
axioms of boom: LEM, propext
independent checker: 53 globals re-checked, problems: []
```

The fault is in the rules of `R`, and not in the encoding of the proof or in the axioms. The kernel of Lean 4.15.0, which applies the usual positivity condition, refuses `R`, and `Q` and the control as well (Listing 8). If the constructor of `R` and the elimination into propositions that its declaration would give are postulated as axioms, Lean proves False from `Classical.propComplete`, the statement that every proposition is equal to True or to False (Listing 9; both Lean listings use the core library only).

Listing 8. Lean 4.15.0 and the three families of Listing 6: its kernel refuses all of them.

```
-- Does Lean 4.15.0 admit the three families of dev_positivity.py?  (B0 is the control.)
inductive B0 : Type where
  | mk0 : ((B0 -> False) -> B0) -> B0

inductive Q : Prop -> Type where
  | mkq (A : Prop) (h : Q (Q A -> False)) : Q A

inductive R : Prop -> Type where
  | mkr (A : Prop) : R (R A -> False)
```

```
dev_lean_decl.lean:2:0: error: (kernel) arg #1 of 'B0.mk0' has a non positive occurrence
    ↪  of the datatypes being declared
dev_lean_decl.lean:5:0: error: (kernel) arg #2 of 'Q.mkq' contains a non valid
    ↪ occurrence of the datatypes being declared
dev_lean_decl.lean:8:0: error: (kernel) invalid return type for 'R.mkr'
```

Listing 9. Lean 4.15.0 with the rules of `R` as axioms: `False` follows from `Classical.propComplete`.

```
-- The rules that the declaration of R would give, postulated as axioms (Lean refuses
    ↪ the declaration).
axiom R : Prop -> Type
axiom mkr : forall A : Prop, R (R A -> False)
axiom R_inv : forall x : Prop, R x -> Exists (fun B : Prop => (R B -> False) = x)

-- Write E x for (R x -> False).
-- K1: if x is a value of E, then E x is refuted.
theorem K1 (B x : Prop) (h : (R B -> False) = x) : Not (R x -> False) :=
  fun e => e (Eq.mp (congrArg R h) (mkr B))

-- K3: if x is not a value of E, then E x holds.
theorem K3 (x : Prop) (hn : Not (Exists (fun B : Prop => (R B -> False) = x))) :
    R x -> False :=
  fun r => hn (R_inv x r)

theorem boom : False := by
  -- Classical.propComplete (a : Prop) : a = True \/ a = False
  have ha := Classical.propComplete (R False -> False)
  have hb := Classical.propComplete (R True -> False)
  cases ha with
  | inl ha =>
    -- E False = True: True is a value of E, so E True is refuted, so E True = False:
    -- False is a value of E, so E False is refuted
    have n1 : Not (R True -> False) := K1 False True ha
    have hb' : (R True -> False) = False :=
      propext (Iff.intro (fun h => n1 h) (fun h => False.elim h))
    have n0 : Not (R False -> False) := K1 True False hb'
    exact n0 (Eq.mpr ha True.intro)
  | inr ha =>
    cases hb with
    | inl hb =>
      -- E True = True: True is a value of E, so E True is refuted
```

```
      have n1 : Not (R True -> False) := K1 True True hb
      exact n1 (Eq.mpr hb True.intro)
    | inr hb =>
      -- E False = False and E True = False: True is not a value of E, so E True holds
      have hni : Not (Exists (fun B : Prop => (R B -> False) = True)) := by
        intro hex
        cases hex with
        | intro B hB =>
          cases Classical.propComplete B with
          | inl hBT =>
            have e : (R B -> False) = (R True -> False) :=
              congrArg (fun P => R P -> False) hBT
            exact Eq.mpr (Eq.trans (Eq.symm (Eq.trans e hb)) hB) True.intro
          | inr hBF =>
            have e : (R B -> False) = (R False -> False) :=
              congrArg (fun P => R P -> False) hBF
            exact Eq.mpr (Eq.trans (Eq.symm (Eq.trans e ha)) hB) True.intro
      exact Eq.mp hb (K3 True hni)

#print axioms boom
```

```
'boom' depends on axioms: [R, R_inv, mkr, propext, Classical.choice, Quot.sound]
```

The axioms are needed for this proof. Together they say that Prop has two elements, True and False, and in a model that interprets Prop by two truth values `R` has no interpretation: E would be a map of $\{0,1\}$ to itself with $E\,x = 1$ exactly when x is not in its image, and none of the four such maps is. Both axioms are standard (Lean's core library has them, excluded middle through its axiom of choice, and the standard library of Coq offers them), and the library of this repository assumes the first. We do not know whether the calculus with `R` and without the axioms is consistent, and we know of no exploit of `Q`.

Exposure. We read back every inductive family of the world of the bank through the kernel's term view (`measure_deviations.py`): 14 families with 19 constructors, of which 3 have a recursive field. None has the family in an index of a recursive field, in a result index, or anywhere else that condition A5 rejects, so the shipped libraries and the statements of the bank are not affected. The scan flags the two families of Listing 6 and none of the families of the natural-number library, which it checks before it reads the bank. The environment can declare neither a family nor an axiom, and the shipped libraries assume excluded middle and not propositional extensionality, so the exploit is open to the author of a library, through the Python interface of the kernel, and to no one else; the libraries, the proofs of the bank and the measurements of this report do not depend on the gap.

Repair. Reject an occurrence of the family in the arguments of a recursive occurrence and in the result indices, in the admission of the family; make the independent checker re-check admission from the stated condition; add the exploit to the gates, with the declaration of `R` now refused. This repair is not applied in this version.

B The reference proofs checked

Table 11 lists the 16 reference proofs of the bank that Figure 5 plots, sampled evenly by length: the number of actions, the size of the proof term counted as a tree, the time of the kernel's check against the statement, the time of the independent checker in a child process with a cap of 120 s and 2,500 MB, the ratio of the two times, and the memory the checker allocated beyond the world it inherited.

C Probes by kind of action

Table 12 gives the latency of probes by the kind of action, for the eleven most frequent of the 17 kinds that the blind search and the random policy of Section 5.2 tried, with the number of probes that ended `ok`, `refused` and `fuel`. A blind search and a random policy try actions in proportions

problem	actions	term nodes	kernel (ms)	indep. (ms)	ratio	MB
aime1986_p4	1	124,359	20.5	2827	138	57
ident_diff_squares	1	4,665	1.30	113	86	1
ineq_am_gm_two	2	9,911	2.70	200	74	2
amc_ineq_01	4	16,887	4.29	302	70	4
ineq_three_squares	4	34,311	8.40	556	66	10
amc_ineq_05	6	20,353	6.09	456	75	7
amc_ind_02	7	9,223	2.63	150	57	2
amc_seq_01	10	44,151	16.9	922	55	15
aime1990_p15	13	96,811	59.6	2973	50	81
no_f_f_succ	16	57,427	22.4	1190	53	17
pell_infinitely_many	19	51,945	26.4	1265	48	35
amc_seq_02	31	157,199	75.8	3302	44	84
imo1977_p6	43	57,243	37.6	1504	40	25
imo1964_p2	53	486,741	159	17469	110	395
amc_prime_01	119	448,062	1448	53648	37	1257
aime2002ii_p3	466	1,444,581	6429	stopped at the memory cap after 99 s		

Table 11. The sample of reference proofs behind Figure 5. The times are those of one run on the machine of Section 5.2.

that a trained policy would not, so the table shows what each kind of probe can cost and not how often it would be asked.

kind	probes	median (ms)	90th (ms)	99th (ms)	ok / refused / fuel
APPLY	866	0.438	2.43	24.9	259 / 384 / 223
CASES	494	0.961	1.70	4.40	410 / 84 / 0
REWRITE	213	0.549	1.40	1.73	161 / 52 / 0
REWRITE_AT	61	0.483	1.21	1.82	0 / 61 / 0
LINARITH	213	0.381	1.21	4.77	12 / 201 / 0
NLINARITH	80	0.964	10.8	62.5	3 / 77 / 0
RING	36	0.533	1.33	3.08	0 / 36 / 0
BY_CONTRA	106	0.285	0.764	2.61	106 / 0 / 0
INDUCT_WITH	48	3.56	11.0	13.0	48 / 0 / 0
CLAIM	40	1.22	6.88	158	36 / 3 / 1
INTERVAL_CASES	12	45.7	73.9	133	12 / 0 / 0

Table 12. Probes by kind of action, from the runs of Table 8.

D The gates

Table 13 lists the 54 gates with the time each took in the run for this report, on the machine of Section 5.2, two gates at a time; the machine also did other work of the report while the gates ran, so the times are indicative. The description is the first clause of the one in `tests/README.md`. All 54 gates pass at commit f840710 (in the run of all the gates the layout gate S45 objected to the untracked directory of this report, as its record in the log shows; the record kept for it here is that of a run of S45 alone, with the directory moved aside). The log of the run, one record per gate, is in the supplementary material.

Table 13. The gates, the time of each in the run for this report, and what it protects.

gate	script	s	protects
S2	`kernel/s2_tests.py`	<1	The full typing core of the hybrid kernel against the reference kernel
S3	`kernel/s3_tests.py`	2	Metas, unification, intro/apply/rewrite against the reference, as lockstep tactic scripts
S4	`kernel/s4_tests.py`	2	Cases and induct against the reference, as lockstep tactic scripts
S5	`kernel/s5_probe_tests.py`	4	The search's probing protocol
S5	`kernel/s5_tests.py`	3	The library index
S6	`kernel/s6_f6_tests.py`	<1	Induction with a Pi-headed goal, and with an explicit generalizing list
S6	`kernel/s6_logic_tests.py`	<1	The logical vocabulary (F7) with a shape probe per connective, and the F5 shape (elimination over a lambda predicate)
S7	`kernel/s7_param_tests.py`	1	The parameter / index distinction (kernel option 'parameters')
S8	`kernel/s8_indep_tests.py`	1	The independently written checker against the kernels
S9	`kernel/s9_strong_tests.py`	<1	F6 probe: strong induction by a BOUNDED PREDICATE (Below P n := True / And (Below P k) (P k)), so the induction ...
S10	`kernel/s10_noconf_tests.py`	<1	Injectivity: generated lemmas are accepted by the kernel's CHECKED path and by the independent checker, and they ...
S11	`kernel/s11_eqgen_tests.py`	<1	Index generalisation with equations (induct on a hypothesis whose index is a concrete term)
S12	`kernel/s12_probe_eq_tests.py`	4	Probing protocol x index generalisation (hybrid only
S13	`kernel/s13_prop_tests.py`	<1	prop_elim: level 0 is Prop, and an inductive family in Type0 that is not a syntactic subsingleton eliminates only ...
S14	`kernel/s14_prop_world_tests.py`	2	The Prop/Type world (prop_elim + parameters, data in Type1): the library built by tactics in this mode, every global ...
S15	`kernel/s15_classical_tests.py`	<1	Classical logic in the Prop/Type world: the four axioms are the only additions to the trusted base
S16	`kernel/s16_quot_tests.py`	<1	Quotients
S17	`kernel/s17_irrelevance_tests.py`	4	Definitional proof irrelevance
S18	`kernel/s18_deep_tests.py`	28	The kernel survives terms of any depth
S19	`kernel/s19_boundary_tests.py`	3	The Python API boundary
S20	`kernel/s20_reservation_tests.py`	<1	The global environment stays WELL FOUNDED
S21	`kernel/s21_view_tests.py`	<1	Reading terms back (Kernel.view) and the validity of term handles

gate	script	s	protects
S26	kernel/s26_memo_tests.py	6	The facts every term node carries, the shared-term (DAG) traversals, the typing memo, and the fast proof extraction
S27	kernel/s27_fuel_tests.py	26	The work budget
S28	kernel/s28_headunfold_tests.py	<1	A definition at the head of a hypothesis or a goal is unfolded ON SYNTAX, so that its arguments stay as the author ...
S29	kernel/s29_apply_pi_goal_tests.py	<1	apply of a lemma to a goal that is itself a Pi (an implication, a negation, a universal statement)
S32	kernel/s32_synconv_tests.py	2	The quick conversion check (SynConv) in front of the evaluator
S33	kernel/s33_convmemo_tests.py	3	Conversion and unification over SHARED structure (the tables ConvMemo and UnifyMemo)
S35	kernel/s35_matchcheck_tests.py	<1	The kernel checks a match against the type of its scrutinee AS WRITTEN, and the result type of the match is read off ...
S39	kernel/s39_failfast_tests.py	1	A comparison that FAILS is decided by what it found, not by normalising both sides
oracle	kernel/oracle_gate.py	18	The self-consistency oracle: random tactic scripts run in lockstep on both kernels and re-checked
S22	library/s22_simp_tests.py	<1	The display normal form (simp_normalize), the change tactic and apply with folded premises
S23	library/s23_ring_tests.py	15	The natural-number library and the ring tactic
S24	library/s24_bin_tests.py	20	The binary natural numbers (bin_lib.py: Pos, N) and their bridge to the unary numerals
S25	library/s25_nring_tests.py	18	ring over the binary naturals N (native, no detour through the unary numerals), and the lemma set it shares with the ...
S30	library/s30_zlib_tests.py	15	The integers Z (int_lib.py)
S31	library/s31_linarith_tests.py	38	linarith and nlinarith over the integers (linarith.py)
S34	library/s34_nt_tests.py	99	Elementary number theory over the integers (nt_lib.py) and the first olympiad problems (imo_lib.py), ...
S37	library/s37_atoms_tests.py	7	Atoms up to the arithmetic in their arguments (ring.py: Ring.canon / canon_arg, shared by ring, linarith, ...
S36	environment/s36_gym_tests.py	16	The proof environment on the hybrid kernel (gym_env.py, gym_search.py), the clear tactic and the observation of goals
S38	environment/s38_terms_tests.py	9	The actions that carry terms (INSTANTIATE, USE), the term pool they draw from, probes with a chosen work budget, ...
S40	environment/s40_actions_tests.py	78	The second action language: the typed term builder (gym_build.py), the actions that say their arguments ...
S42	environment/s42_pins_tests.py	4	Pins: small direct tests of the corner cases of the typed term builder (gym_build.py) and of the action language ...
S47	environment/s47_hindsight_tests.py	278	Hindsight: the proof tree of an episode (gym_tree.py) and the theorems it proved

gate	script	s	protects
S41	`problems/s41_bank_tests.py`	90	The problem bank
S43	`problems/s43_bank2_tests.py`	477	The extended problem bank
S49	`problems/s49_contest_tests.py`	209	The contest problems (AMC/AIME/ISL/IMO/APMO and classical exercises): provenance (origin, verified), statements that ...
S50	`problems/s50_proofs_tests.py`	1107	The proofs of the contest problems: the premises mask, every recorded proof replayed in a fresh engine and ...
harness	`support/harness.py`	<1	Differential harness: reference kernel (absolute levels) vs hybrid kernel (bound = index, context = level)
S44	`learner/s44_ml_tests.py`	189	The learner's machinery
S51	`learner/s51_suite_tests.py`	690	The training suite and the experiment layer
S48	`synth/s48_synth_tests.py`	158	The synthetic theorem generator (src/python/synth, docs/GENERATOR.md) and its datasets
S45	`repo/s45_layout_tests.py`	75	The repository's layout is the layout the documentation describes, and stays so
S46	`repo/s46_examples_tests.py`	377	The examples run (example 06 is expected to say SKIPPED when gymnasium is not installed)

References

[1] A. Abel and B. Pientka. Higher-order dynamic pattern unification for dependent types and records. In *Typed Lambda Calculi and Applications (TLCA)*, volume 6690 of *Lecture Notes in Computer Science*, pages 10–26. Springer, 2011. doi: 10.1007/978-3-642-21691-6_5.

[2] A. Abel, J. Öhman, and A. Vezzosi. Decidability of conversion for type theory in type theory. *Proceedings of the ACM on Programming Languages*, 2(POPL), 2018.

[3] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[4] L. Aniva, C. Sun, B. Miranda, C. Barrett, and S. Koyejo. Pantograph: a machine-to-machine interaction interface for advanced theorem proving, high level reasoning, and data extraction in Lean 4. arXiv:2410.16429, 2024.

[5] K. Bansal, S. M. Loos, M. N. Rabe, C. Szegedy, and S. Wilcox. HOList: an environment for machine learning of higher-order logic theorem proving. In *International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 454–463, 2019.

[6] H. P. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[7] U. Berger and H. Schwichtenberg. An inverse of the evaluation functional for typed λ-calculus. In *Symposium on Logic in Computer Science (LICS)*, pages 203–211. IEEE Computer Society, 1991.

[8] R. G. Bland. New finite pivoting rules for the simplex method. *Mathematics of Operations Research*, 2(2):103–107, 1977.

[9] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. arXiv:1606.01540, 2016.

[10] M. M. Carneiro. The type theory of Lean. Master's thesis, Carnegie Mellon University, 2019. `https://github.com/digama0/lean-type-theory`.

[11] M. M. Carneiro. Lean4Lean: Verifying a typechecker for Lean, in Lean. arXiv:2403.14064, 2024.

[12] A. Charguéraud. The locally nameless representation. *Journal of Automated Reasoning*, 49(3): 363–408, 2012. doi: 10.1007/s10817-011-9225-2.

[13] J. Cockx, D. Devriese, and F. Piessens. Pattern matching without K. In *International Conference on Functional Programming (ICFP)*, pages 257–268. ACM, 2014. doi: 10.1145/2628136.2628139.

[14] T. Coquand. An analysis of Girard's paradox. In *Symposium on Logic in Computer Science (LICS)*, pages 227–236. IEEE Computer Society, 1986.

[15] T. Coquand. Metamathematical investigations of a calculus of constructions. In P. Odifreddi, editor, *Logic and Computer Science*, pages 91–122. Academic Press, 1990.

[16] T. Coquand and G. Huet. The calculus of constructions. *Information and Computation*, 76(2–3): 95–120, 1988.

[17] T. Coquand and C. Paulin-Mohring. Inductively defined types. In *COLOG-88: International Conference on Computer Logic*, volume 417 of *Lecture Notes in Computer Science*, pages 50–66. Springer, 1990. doi: 10.1007/3-540-52335-9_47.

[18] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae*, 34: 381–392, 1972. doi: 10.1016/1385-7258(72)90034-0.

[19] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE-28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[20] M. Dos Santos et al. Kimina Lean Server: a high-performance Lean server for large-scale verification. arXiv:2504.21230, 2025.

[21] P. Dybjer. Inductive families. *Formal Aspects of Computing*, 6(4):440–465, 1994. doi: 10.1007/BF01211308.

[22] J.-C. Filliâtre and S. Conchon. Type-safe modular hash-consing. In *ACM Workshop on ML*, pages 12–19. ACM, 2006.

[23] G. Gilbert, J. Cockx, M. Sozeau, and N. Tabareau. Definitional proof-irrelevance without K. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019. doi: 10.1145/3290316.

[24] E. Giménez. Codifying guarded definitions with recursive schemes. In *Types for Proofs and Programs (TYPES 1994)*, volume 996 of *Lecture Notes in Computer Science*, pages 39–59. Springer, 1995.

[25] H. Goguen, C. McBride, and J. McKinna. Eliminating dependent pattern matching. In *Algebra, Meaning, and Computation: Essays Dedicated to Joseph A. Goguen on the Occasion of His 65th Birthday*, volume 4060 of *Lecture Notes in Computer Science*, pages 521–540. Springer, 2006. doi: 10.1007/11780274_27.

[26] B. Grégoire and X. Leroy. A compiled implementation of strong reduction. In *International Conference on Functional Programming (ICFP)*, pages 235–246. ACM, 2002. doi: 10.1145/581478.581501.

[27] B. Grégoire and A. Mahboubi. Proving equalities in a commutative ring done right in Coq. In *Theorem Proving in Higher Order Logics (TPHOLs)*, volume 3603 of *Lecture Notes in Computer Science*, pages 98–113. Springer, 2005. doi: 10.1007/11541868_7.

[28] J. M. Han, J. Rute, Y. Wu, E. W. Ayers, and S. Polu. Proof artifact co-training for theorem proving with language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[29] J. Harrison. Verifying nonlinear real formulas via sums of squares. In *Theorem Proving in Higher Order Logics (TPHOLs)*, volume 4732 of *Lecture Notes in Computer Science*, pages 102–118. Springer, 2007.

[30] M. Hofmann. A simple model for quotient types. In *Typed Lambda Calculi and Applications (TLCA)*, volume 902 of *Lecture Notes in Computer Science*, pages 216–234. Springer, 1995.

[31] D. Huang, P. Dhariwal, D. Song, and I. Sutskever. GamePad: a learning environment for theorem proving. In *International Conference on Learning Representations (ICLR)*, 2019.

[32] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[33] T. Hubert, R. Mehta, L. Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.

[34] G. P. Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1(1):27–57, 1975.

[35] A. J. C. Hurkens. A simplification of Girard's paradox. In *Typed Lambda Calculi and Applications (TLCA)*, volume 902 of *Lecture Notes in Computer Science*, pages 266–278. Springer, 1995.

[36] W. Jakob. nanobind: tiny and efficient C++/Python bindings. `https://github.com/wjakob/nanobind`, 2022.

[37] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample. Draft, sketch, and prove: guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations (ICLR)*, 2023.

[38] G. Lample, M.-A. Lachaux, T. Lavril, X. Martinet, A. Hayat, G. Ebner, A. Rodriguez, and T. Lacroix. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

[39] Y. Lin, S. Tang, B. Lyu, et al. Goedel-Prover: a frontier model for open-source automated theorem proving. arXiv:2502.07640, 2025.

[40] C. McBride and J. McKinna. Functional pearl: I am not a number – I am a free variable. In *Haskell Workshop*. ACM, 2004.

[41] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4):497–536, 1991.

[42] E. W. Myers. An applicative random-access stack. *Information Processing Letters*, 17(5):241–248, 1983.

[43] T. Nipkow. Functional unification of higher-order patterns. In *Symposium on Logic in Computer Science (LICS)*, pages 64–74. IEEE Computer Society, 1993.

[44] C. Okasaki. Purely functional random-access lists. In *Functional Programming Languages and Computer Architecture (FPCA)*, pages 86–95. ACM, 1995.

[45] OpenAI. lean-gym: a Lean 3 interface to interact with proof states. GitHub repository, `https://github.com/openai/lean-gym`, 2022.

[46] C. Paulin-Mohring. Inductive definitions in the system Coq – rules and properties. In *Typed Lambda Calculi and Applications (TLCA)*, volume 664 of *Lecture Notes in Computer Science*, pages 328–345. Springer, 1993.

[47] G. Poesia and N. D. Goodman. Peano: learning formal mathematical reasoning. *Philosophical Transactions of the Royal Society A*, 2023. arXiv:2211.15864.

[48] G. Poesia, D. Broman, N. Haber, and N. D. Goodman. Learning formal mathematics from intrinsic motivation. arXiv:2407.00695, 2024.

[49] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. arXiv:2009.03393, 2020.

[50] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever. Formal mathematics statement curriculum learning. In *International Conference on Learning Representations (ICLR)*, 2023.

[51] Z. Z. Ren, Z. Shao, J. Song, et al. DeepSeek-Prover-V2: advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. arXiv:2504.21801, 2025.

[52] M. Sozeau, S. Boulier, Y. Forster, N. Tabareau, and T. Winterhalter. Coq Coq Correct! verification of type checking and erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL), 2020. doi: 10.1145/3371076.

[53] The mathlib Community. The Lean mathematical library. In *Certified Programs and Proofs (CPP)*, pages 367–381. ACM, 2020.

[54] The Rocq Development Team. The Rocq prover (formerly Coq). `https://rocq-prover.org`, 2025.

[55] M. Towers, A. Kwiatkowski, J. Terry, et al. Gymnasium: a standard interface for reinforcement learning environments. arXiv:2407.17032, 2024.

[56] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[57] Y. Wu, A. Q. Jiang, J. Ba, and R. Grosse. INT: an inequality benchmark for evaluating generalization in theorem proving. In *International Conference on Learning Representations (ICLR)*, 2021.

[58] H. Xin, D. Guo, Z. Shao, et al. DeepSeek-Prover: advancing theorem proving in LLMs through large-scale synthetic data. arXiv:2405.14333, 2024.

[59] H. Xin, Z. Z. Ren, J. Song, et al. DeepSeek-Prover-V1.5: harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. arXiv:2408.08152, 2024.

[60] K. Yang and J. Deng. Learning to prove theorems via interacting with proof assistants. In *International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 6984–6994, 2019.

[61] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar. LeanDojo: theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023.

[62] K. Zheng, J. M. Han, and S. Polu. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

[63] B. Ziliani and M. Sozeau. A unification algorithm for Coq featuring universe polymorphism and overloading. In *International Conference on Functional Programming (ICFP)*, pages 179–191. ACM, 2015.